

Zaawansowane Techniki Internetowe

Sprawozdanie z projektu

System wypożyczalni samochodów

DriveEase

Autorzy Michał Kowalik | nr albumu 415453

Kamil Pszeniczka | nr albumu 414342

Kierunek ITE

Stopień / Rok / Semestr II stopień, rok 1, semestr 1

Grupa 3

Data 01.06.2026

Spis treści

1. Podział zadań	2
2. Cel projektu	2
3. Użyte technologie	2
Frontend	2
Backend	2
Bazy danych (persystencja poliglotyczna)	3
Infrastruktura i DevOps	3
4. Zakładane funkcjonalności	3
5. Architektura aplikacji	4
Warstwy backendu	4
Cykl życia żądania	4
6. Struktura plików	5
7. Projekt bazy danych	5
7.1. PostgreSQL — model relacyjny	6
7.2. MongoDB — model dokumentowy	9
7.3. Redis — model key-value	10
7.4. Migracje (Alembic)	11
8. Implementacja — backend	12
8.1. Modele danych (warstwa ORM)	12
8.2. Uwierzytelnianie i bezpieczeństwo	13
8.3. Dynamiczna wycena i scoring ryzyka	16
8.4. Wynajem: odbiór, zwrot i historia	18
8.5. System recenzji	19
8.6. Maile transakcyjne	20
9. Implementacja — frontend	21
9.1. Ochrona tras (middleware)	22
9.2. Kontekst sesji (AuthContext)	22
9.3. Pobieranie danych (SWR) — wycena spięta z backendem	23
9.4. Mutacje i spójność cache	23
9.5. Motyw jasny/ciemny i wielojęzyczność	24
9.6. Panele według ról	25
9.7. Logika domenowa po stronie klienta (czyste funkcje)	26
10. Katalog endpointów API	27
Uwierzytelnianie — <code>/auth</code>	27
Profil — <code>/users</code>	28
Katalog — <code>/vehicles</code> , <code>/categories</code> , <code>/pricing</code>	28
Rezerwacje i wynajmy — <code>/reservations</code> , <code>/rentals</code>	28
Recenzje — <code>/reviews</code>	28
Serwis — <code>/service-orders</code> , <code>/vehicles/{id}/service-orders</code>	29
Administracja — <code>/admin</code> , <code>/admin/vehicles</code> , <code>/admin/customers</code>	29
11. Wzorce i decyzje projektowe	30
12. Bezpieczeństwo	31
13. Przepływy użytkownika	31
14. Testy	32
15. Wnioski	33
16. Instrukcja uruchomienia aplikacji	34
Wariant A — Docker Compose (zalecany)	34
Wariant B — uruchomienie lokalne (dev)	34
17. Spis listingów i rysunków	35

1. Podział zadań

Projekt realizowany był w dwuosobowym zespole z wyraźnym podziałem:

- **Michał Kowalik — frontend + CI/CD:** aplikacja kliencka w Next.js (React, TypeScript), komponenty UI, panele zależne od roli, motyw jasny/ciemny, wielojęzyczność (PL/EN), warstwa hooków SWR do komunikacji z API, ochrona tras (middleware), a także konfiguracja potoku ciągłej integracji (GitHub Actions), konteneryzacja frontendu i reverse-proxy (nginx).
- **Kamil Pszeniczka — backend + bazy danych:** API w FastAPI, modele i migracje (PostgreSQL / SQLAlchemy / Alembic), logika domenowa (wyceny, scoring ryzyka, rezerwacje, wynajmy, recenzje, serwis), uwierzytelnianie JWT, cache i tokeny w Redisie, logi i recenzje w MongoDB, wysyłka maili transakcyjnych oraz testy jednostkowe i integracyjne backendu.

2. Cel projektu

Celem projektu było zaprojektowanie i implementacja kompletnej, wielowarstwowej aplikacji webowej wypożyczalni samochodów, łączącej publiczny katalog pojazdów z rozbudowanym zapleczem operacyjnym (panele pracownika, serwisanta i administratora). Aplikacja miała zademonstrować praktyczne zastosowanie nowoczesnego stosu technologii internetowych: oddzielnego frontendu (SPA/SSR), REST API, **wielu komplementarnych baz danych** (relacyjna + dokumentowa + key-value) oraz pełnej konteneryzacji środowiska.

Kluczowym, wyróżniającym elementem merytorycznym jest **dynamiczna cena wynajmu** zależna od profilu ryzyka klienta — system automatycznie klasyfikuje użytkowników na podstawie historii wynajmów i zgłoszonych incydentów, a następnie nalicza zniżkę lojalnościową lub dopłatę za ryzyko. Dodatkowymi celami były: bezpieczne uwierzytelnianie (JWT, weryfikacja e-mail, reset hasła z ochroną przed nadużyciami), system recenzji pojazdów oraz dbałość o jakość kodu (lintery, statyczna kontrola typów, testy jednostkowe i end-to-end w potoku CI).

Z punktu widzenia dydaktycznego projekt miał pokazać:

- separację odpowiedzialności (warstwy: prezentacja → API → logika → dane),
- pracę z **trzema różnymi modelami danych** i świadomy dobór bazy do rodzaju informacji,
- asynchroniczny backend (FastAPI + SQLAlchemy async) obsługujący równoległe żądania,
- automatyzację jakości (CI/CD) i konteneryzację (Docker Compose).

3. Użyte technologie

Frontend

- **Next.js 16** (App Router) + **React 19**, **TypeScript 5**
- **Tailwind CSS 4** + **shadcn/ui** (Radix UI) — system komponentów dostępnościowych
- **SWR** — pobieranie danych, cache i rewalidacja po stronie klienta (warstwa hooków `use*`)
- **lucide-react** — ikony; własny *middleware* ochrony tras (`src/proxy.ts`)
- **Jest** + **ts-jest** — testy jednostkowe; **ESLint** + **Prettier** — jakość kodu

Backend

- **FastAPI 0.115** (Python 3.12), serwer **Uvicorn**
- **SQLAlchemy 2.0** (tryb asynchroniczny) + sterownik **asyncpg**, migracje **Alembic**
- **Motor** — asynchroniczny klient MongoDB; **redis-py** (async) — Redis
- **python-jose** (JWT), **passlib** + **bcrypt** (hashowanie haseł)
- **Pydantic v2** / **pydantic-settings** — walidacja i konfiguracja; **Pillow** — obróbka zdjęć
- **pytest** + **pytest-asyncio** + **httpx** — testy; **Ruff** (lint/format), **mypy** (typy)

Bazy danych (persystencja poliglotyczna)

- **PostgreSQL 17** — dane transakcyjne (użytkownicy, pojazdy, rezerwacje, wynajmy, rozbicia cen, incydenty, serwis)
- **MongoDB 7** — logi odbioru/zwrotu pojazdu (zdjęcia, podpis) oraz recenzje
- **Redis 7** — cache modelu użytkownika, blacklista tokenów JWT, jednorazowe tokeny (weryfikacja e-mail, reset hasła) z TTL

Infrastruktura i DevOps

- **Docker + Docker Compose** — uruchomienie całego środowiska jedną komendą
- **nginx (alpine)** — reverse proxy: / → frontend, /api → backend, serwowanie awatarów
- **Mailpit** — lokalny serwer SMTP do podglądu maili; **pgAdmin** — podgląd bazy PostgreSQL
- **GitHub Actions** — CI: lint + type-check + testy dla frontendu i backendu (z detekcją zmian)
- **Playwright** — testy end-to-end (e2e/)

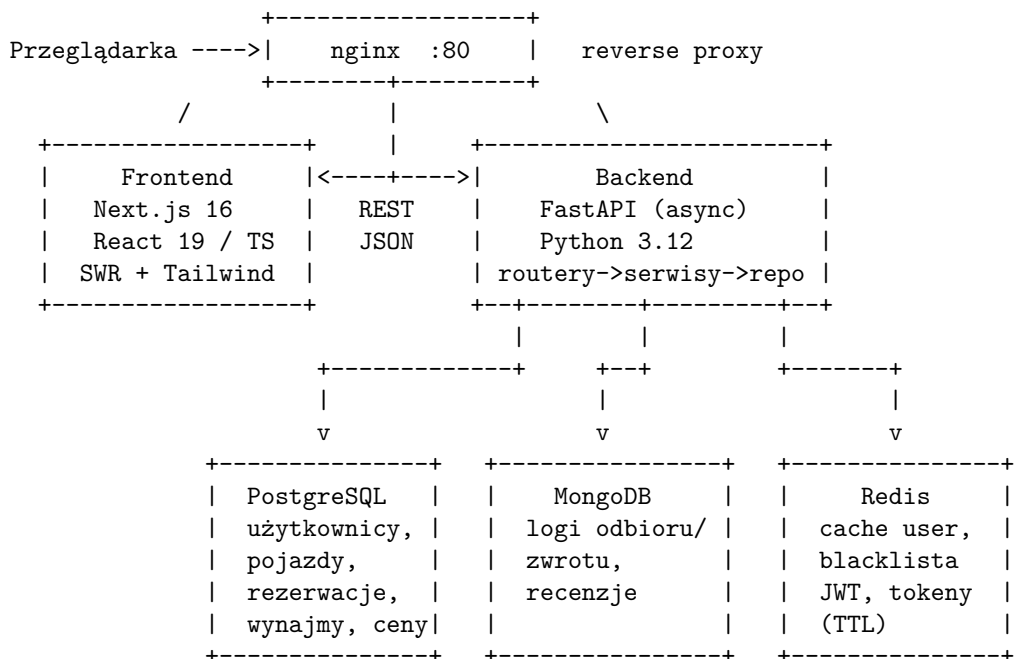
4. Zakładane funkcjonalności

1. **System logowania i kont** — rejestracja, weryfikacja adresu e-mail, logowanie (JWT), odświeżanie i unieważnianie tokenów, „nie pamiętam hasła” / reset hasła.
2. **Katalog dostępnych pojazdów** — filtrowanie (cena, liczba miejsc, rok, typ silnika, dostępność), sortowanie, paginacja, karta pojazdu ze zdjęciami, specyfikacją i kalendarzem.
3. **Panele według ról** — klient, pracownik (employee), serwisant (technician), administrator; nawigacja i widoki zależne od roli.
4. **Rezerwacje i wynajmy** — kreator rezerwacji, potwierdzanie przez pracownika, proces odbioru (pickup) i zwrotu (return) z naliczeniem ceny finalnej.
5. **Dynamiczna cena wynajmu** zależna od **klasyfikacji ryzyka klienta** (zniżka lojalnościowa ↔ dopłata za ryzyko) oraz kategorii pojazdu.
6. **Historia wynajmów** użytkownika oraz historia operacji (logi w MongoDB).
7. **System opinii (recenzji)** — recenzja tylko zakończonego wynajmu, agregacja ocen pojazdu, moderacja przez administratora.
8. **Motyw jasny/ciemny** oraz **wielojęzyczność** (PL/EN).
9. **Maile transakcyjne** — weryfikacja konta, reset hasła, potwierdzenie rezerwacji.
10. **Panel serwisowy** — zlecenia serwisowe i historia serwisowa pojazdów, rejestr incydentów i wewnętrzne notatki o kliencie.

Zmiana względem pierwotnego założenia. Pierwotny pomysł ceny zależnej od *cen paliw/energii z publicznego API* zastąpiono bardziej wartościowym merytorycznie **silnikiem oceny ryzyka klienta** (rozdz. 8). Typ silnika pojazdu jest nadal przechowywany (`engine_type`), lecz nie napędza ceny przez API zewnętrzne. Uzasadnienie — w *Wnioskach* (rozdz. 15).

5. Architektura aplikacji

Architektura jest trójwarstwowa, oparta o **separację warstw** (prezentacja → API → logika → dane), z reverse-proxy nginx jako pojedynczym punktem wejścia i **trzema bazami danych** dobranymi do charakteru przechowywanych informacji.



Narzędzia pomocnicze: Mailpit (SMTP, podgląd maili) - pgAdmin (podgląd PG)

Warstwy backendu

Backend stosuje **warstwowy podział odpowiedzialności**, co ułatwia testowanie i utrzymanie:

Warstwa	Katalog	Odpowiedzialność
Routery	app/routers/	definicja endpointów REST, walidacja wejścia, autoryzacja (Depends)
Serwisy	app/services/	logika domenowa (wyceny, ryzyko, rezerwacje, recenzje, serwis)
Repozytoria	app/repositories/	dostęp do danych (zapytania SQLAlchemy / Mongo)
Modele	app/models/	mapowanie ORM (tabele, ograniczenia, relacje)
Schematy	app/schemas/	kontrakty wejścia/wyjścia (Pydantic)
Rdzeń	app/core/	bezpieczeństwo (JWT, bcrypt), zależności, cache, e-mail

Cykl życia żądania

Typowe uwierzytelnione żądanie przechodzi następującą ścieżkę:

- nginx** kieruje żądanie `/api/...` do backendu (a `/...` do frontendu Next.js).
- FastAPI** dopasowuje router i uruchamia zależności (**Depends**): wyciągnięcie tokenu JWT (z nagłówka lub ciasteczka `httpOnly`), sprawdzenie blacklisty w Redisie, dekodowanie i pobranie użytkownika (najpierw cache Redis, potem PostgreSQL).
- Router** woła odpowiedni **serwis**, który realizuje logikę domenową, korzystając z **repozytoriów** (PostgreSQL/Mongo) i ewentualnie Redisa.

4. Wynik jest serializowany przez **schemat Pydantic** i zwracany jako JSON.

Połączenia do baz otwierane są raz na start procesu (**lifespan**) i zwalniane przy zamknięciu; pula połączeń SQLAlchemy żyje tyle co aplikacja.

6. Struktura plików

Backend (car-rental-backend/):

```
app/
main.py          - punkt wejścia FastAPI: CORS, montaż statyki i routerów, lifespan
config.py        - ustawienia wczytywane z .env (Pydantic Settings)
core/
  security.py     - JWT (python-jose) + hashowanie haseł (bcrypt)
  deps.py         - zależności: get_current_user, require_roles
  email.py        - maile transakcyjne (SMTP)
  token_blacklist.py / user_cache.py - operacje na Redisie
  exceptions.py   - wyjątki domenowe
db/
  - silniki połączeń (engine, mongodb, redis, session)
models/
  - modele ORM (11 tabel)
schemas/
  - kontrakty Pydantic
repositories/
  - dostęp do danych
routers/
  - endpointy REST (13 routerów)
services/
  - logika domenowa (auth, pricing, risk_scoring, rental, review, ...)
alembic/
  - migracje schematu PostgreSQL
scripts/
  - seed.py (dane przykładowe), init_mongo.py (indeksy)
tests/
  - testy pytest
Dockerfile, requirements.txt
```

Frontend (car-rental-frontend/):

```
src/
app/
  - trasy App Routera: /, /register, /verify-email, /dashboard/*
components/
  - komponenty UI (auth, vehicles, bookings, reviews, fleet, customers, ...)
contexts/
  - AuthContext (sesja), SettingsContext (motyw + język)
hooks/
  - hooki SWR (use*) - pobieranie danych i mutacje
i18n/
  - translations.ts (słowniki PL/EN), useTranslation.ts
lib/
  - czyste funkcje (filters, availability, password, formatters)
  __tests__/
    - testy jednostkowe Jest
types/
  - typy domenowe + mapery API→UI
proxy.ts
  - middleware ochrony tras (redirecty wg sesji)
jest.config.js, package.json, Dockerfile
```

7. Projekt bazy danych

Zastosowano **persystencję poliglotyczną** — trzy bazy o różnych modelach danych. Pełne diagramy ERD znajdują się w katalogu docs/ jako pliki PlantUML (erd-postgres.puml, erd-mongo.puml, erd-redis.puml).

7.1. PostgreSQL — model relacyjny

Baza relacyjna przechowuje dane transakcyjne. Integralność wymuszają klucze obce, ograniczenia CHECK, indeksy oraz **częściowe indeksy unikalne** (WHERE ...). Poniżej opis najważniejszych tabel.

Tabela users — konta i role.

Kolumna	Typ	Uwagi
id	uuid	PK
email	varchar(255)	unikalny
hashed_password	text	bcrypt
first_name / last_name	varchar(100)	
role	enum	customer / employee / technician / admin
is_active / is_verified	bool	aktywność konta / weryfikacja e-mail
phone / avatar_url	varchar / text	opcjonalne
risk_score	numeric(5,2)	CK 0..100 — profil ryzyka
last_login_at	timestampz	

Tabela categories — kategorie cenowe pojazdu.

Kolumna	Typ	Uwagi
id	uuid	PK
name	enum	economy / comfort / premium / suv / van (unikalny)
description	text	opcjonalny
price_multiplier	numeric(5,3)	mnożnik ceny bazowej

Tabela vehicles — pojazdy w katalogu.

Kolumna	Typ	Uwagi
id	uuid	PK
category_id	uuid	FK → categories
brand / model	varchar(100)	
year	int	
license_plate / vin	varchar	unikalne częściowo (WHERE is_active)
engine_type	enum	petrol / diesel / electric / hybrid
horsepower / seats / trunk_capacity / mileage	int	CK (> 0 lub ≥ 0)
daily_base_price	numeric(10,2)	cena bazowa za dobę
color / status	enum	status: available / rented / maintenance / out_of_service
is_active	bool	soft-delete
avg_rating / ratings_count	numeric(3,2) / int	zdenormalizowane agregaty ocen

Tabela vehicle_images — zdjęcia pojazdu (1:N). Częściowy indeks unikalny gwarantuje dokładnie jedno zdjęcie główne na pojazd (WHERE is_primary = true); pola: url, position, is_primary.

Tabela reservations — zamówienia klienta.

Kolumna	Typ	Uwagi
id	uuid	PK
user_id / vehicle_id	uuid	FK
start_date / end_date	timestampz	okres najmu
status	enum	pending / confirmed / active / completed / cancelled
total_price	numeric(10,2)	cena wstępna

Tabela rentals — faktyczne wydanie pojazdu (powstaje przy odbiorze). Relacja 1:1 z rezerwacją (reservation_id unikalny). Pola: pickup_date, return_date (CK > pickup), mileage_start/end, fuel_level_start/end (CK 0..100), damage_notes, employee_id (FK).

Tabela rental_price_breakdowns — rozbiecie ceny finalnej (1:1 z rentals): base_price, risk_multiplier (numeric(6,4)), final_price, calculated_at (wszystkie CK ≥ 0).

Tabela incidents — incydenty klienta (uszkodzenie, opóźniony zwrot, mandat, skarga). Pola: customer_id (FK, CASCADE), opcjonalny rental_id (FK, SET NULL), reported_by_id, type (enum), severity (minor / moderate / major), title, description, opcjonalny cost. Severity wpływa na risk_score.

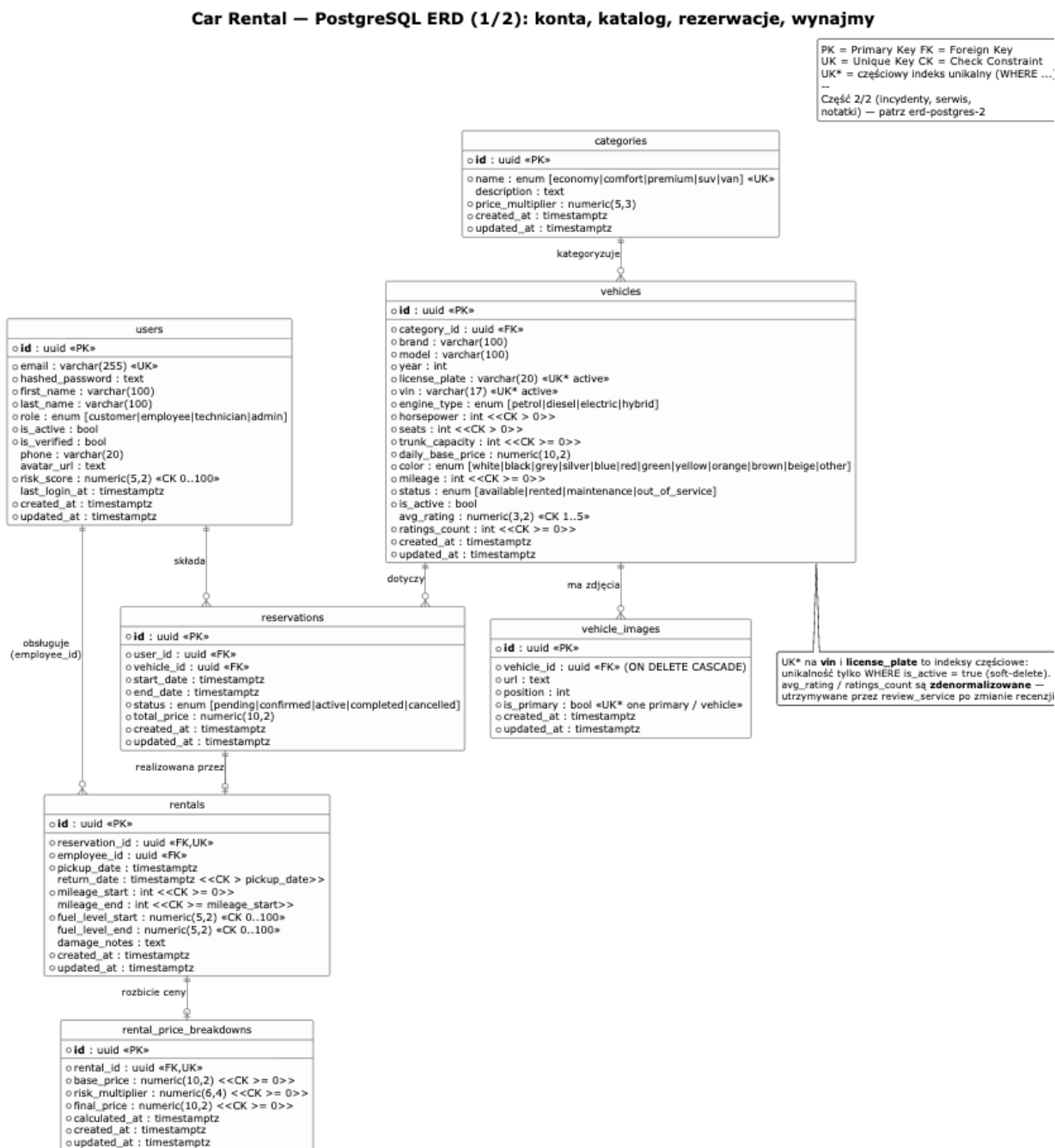
Tabela service_orders — zlecenia serwisowe (cykl scheduled → in_progress → completed). Pola: vehicle_id (FK, CASCADE), type (inspection / repair / tire_swap / wash), status, description, opcjonalny cost, scheduled_date, completed_date, technician_id (FK).

Tabela service_history — wpisy historii serwisu (1:N do zlecenia). Pola: vehicle_id, service_order_id, notes, parts_replaced (natywna tablica TEXT[] w PostgreSQL), mileage_at_service.

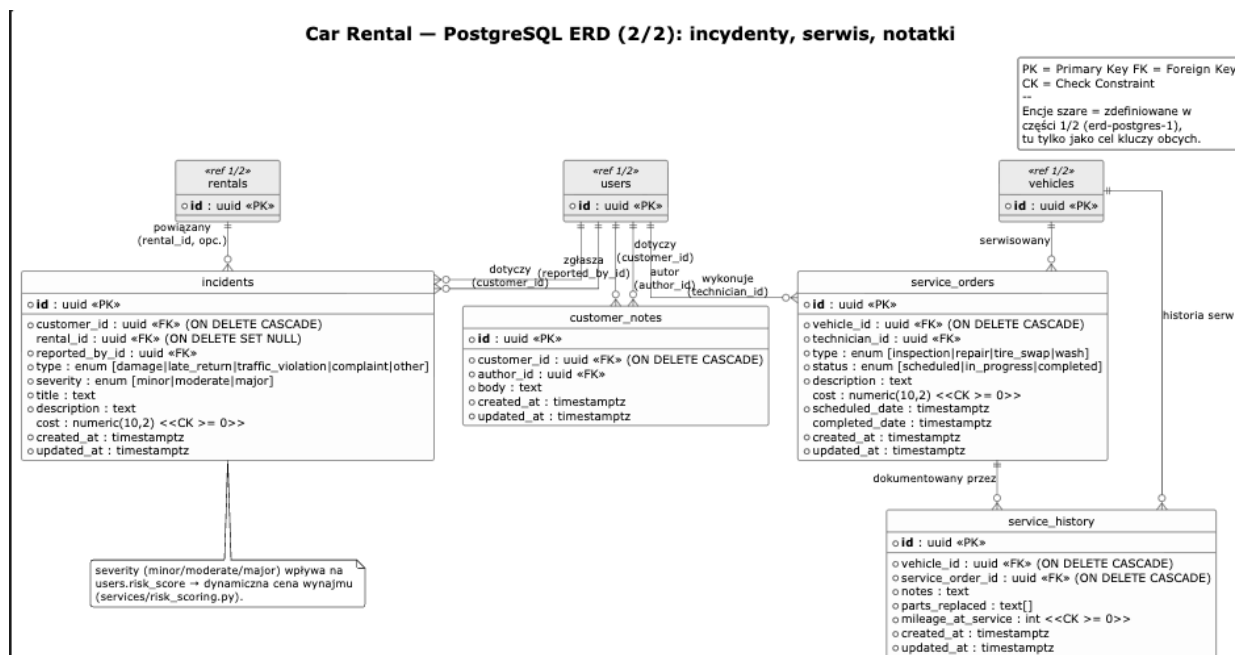
Tabela customer_notes — wewnętrzne notatki pracownika o kliencie (customer_id, author_id, body), niewidoczne dla samego klienta.

Wszystkie tabele dziedziczą po wspólnej klasie bazowej pola id (uuid), created_at i updated_at (timestampz).

Rysunek 1a. Diagram ERD PostgreSQL — część 1/2: konta, katalog, rezerwacje, wynajmy (render docs/erd-postgres.puml).



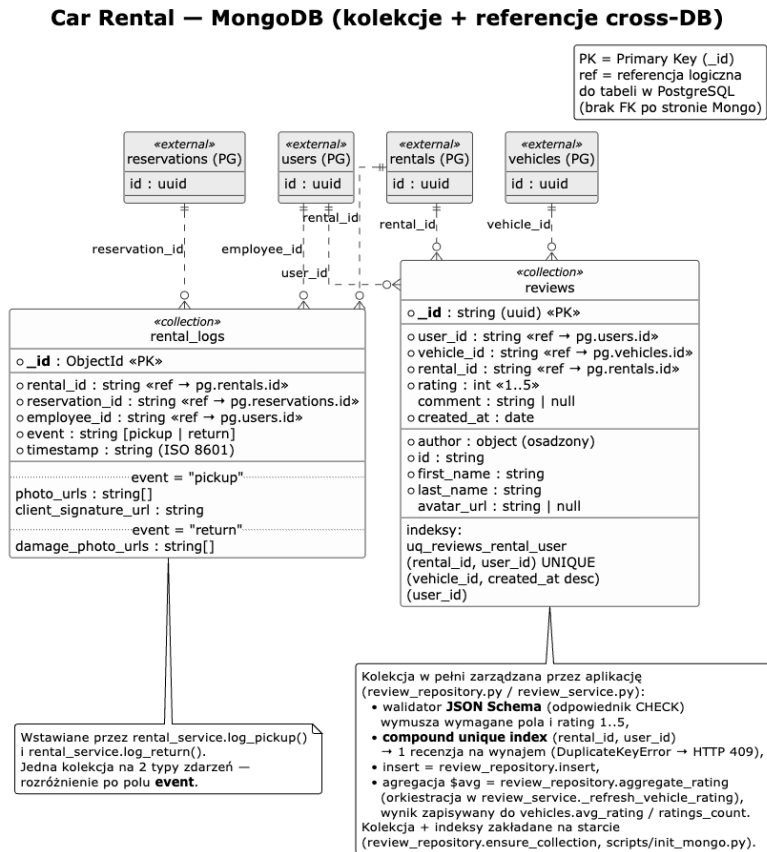
Rysunek 1b. Diagram ERD PostgreSQL — część 2/2: incydenty, serwis, notatki (render docs/erd-postgres.puml).



7.2. MongoDB — model dokumentowy

Dwie kolekcje:

- **rental_logs** — zdarzenia pickup / return z referencjami (jako stringi) do PostgreSQL: rental_id, reservation_id, employee_id, event, timestamp oraz pola zależne od typu zdarzenia (photo_urls, client_signature_url lub damage_photo_urls).
- **reviews** — recenzje pojazdów; **złożony indeks unikalny** (rental_id, user_id) gwarantuje na poziomie bazy maksymalnie jedną opinię na wynajem. Dokument zawiera m.in. user_id, vehicle_id, rental_id, rating, comment, created_at i osadzonego autora.

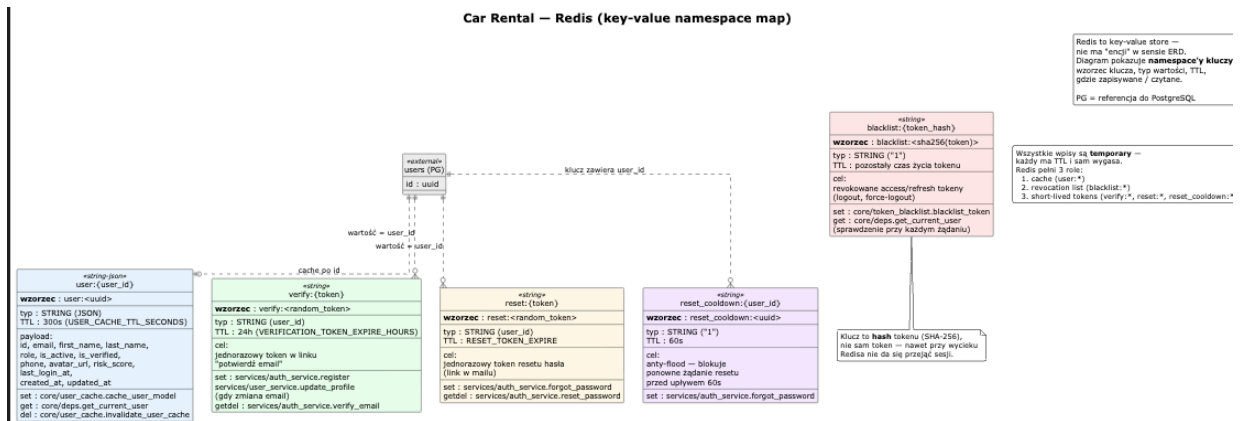
Rysunek 2. Diagram kolekcji MongoDB i referencji cross-DB (render docs/erd-mongo.puml).

7.3. Redis — model key-value

Redis pełni trzy role, wszystkie wpisy mają TTL i wygasają samoczynnie:

Wzorzec klucza	Typ	TTL	Rola
user:{id}	JSON	300 s	cache modelu użytkownika (hot-path autoryzacji)
blacklist:{sha256(token)}	string	do końca życia tokenu	unieważnione tokeny JWT
verify:{token}	string (user_id)	24 h	jednorazowy token weryfikacji e-mail
reset:{token}	string (user_id)	1 h	jednorazowy token resetu hasła
reset_cooldown:{id}	string	60 s	anty-flood resetu hasła

Rysunek 3. Mapa przestrzeni nazw kluczy Redis (render docs/erd-redis.puml).



7.4. Migracje (Alembic)

Schemat PostgreSQL wersjonowany jest migracjami Alembic (łańcuch 12 rewizji: od pustej linii bazowej, przez dodanie kategorii, rezerwacji i wynajmów, pól profilu, zdjęć/incydentów/notatek, zleceń serwisowych, agregatów ocen, aż po normalizację i usunięcie prototypowej dopłaty paliwowej). Migracje uruchamia się komendą `alembic upgrade head`.

8. Implementacja — backend

8.1. Modele danych (warstwa ORM)

Encje opisane są jako modele SQLAlchemy. Poniżej model użytkownika z wyliczeniem ról oraz polem `risk_score`, które jest fundamentem dynamicznej wyceny.

Listing 1. Model User i role (`app/models/user.py`)

```
class UserRole(enum.StrEnum):
    # role sterują autoryzacją
    ↪ (require_roles)
    CUSTOMER = "customer"
    EMPLOYEE = "employee"
    TECHNICIAN = "technician"
    ADMIN = "admin"

class User(Base):
    __tablename__ = "users"
    __table_args__ = (
        # twarde ograniczenie po stronie bazy - score zawsze w przedziale [0, 100]
        CheckConstraint("risk_score >= 0 AND risk_score <= 100",
            name="ck_user_risk_score_range"),
    )

    email: Mapped[str] = mapped_column(String(255), unique=True) # login =
    ↪ unikalny e-mail
    hashed_password: Mapped[str] = mapped_column(Text) # tylko hash
    ↪ bcrypt, nigdy jawne hasło
    first_name: Mapped[str] = mapped_column(String(100))
    last_name: Mapped[str] = mapped_column(String(100))
    role: Mapped[UserRole] = mapped_column(
        Enum(UserRole, native_enum=False), default=UserRole.CUSTOMER, index=True) #
    ↪ domyślnie klient
    is_active: Mapped[bool] = mapped_column(Boolean, default=True) # konto
    ↪ włączone / zablokowane
    is_verified: Mapped[bool] = mapped_column(Boolean, default=False) # czy e-mail
    ↪ potwierdzony
    # profil ryzyka 0..100 - podstawa dynamicznej ceny; indeks, bo czytany przy każdej
    ↪ wycenie
    risk_score: Mapped[Decimal] = mapped_column(Numeric(5, 2), default=Decimal("0.00"),
    ↪ index=True)
```

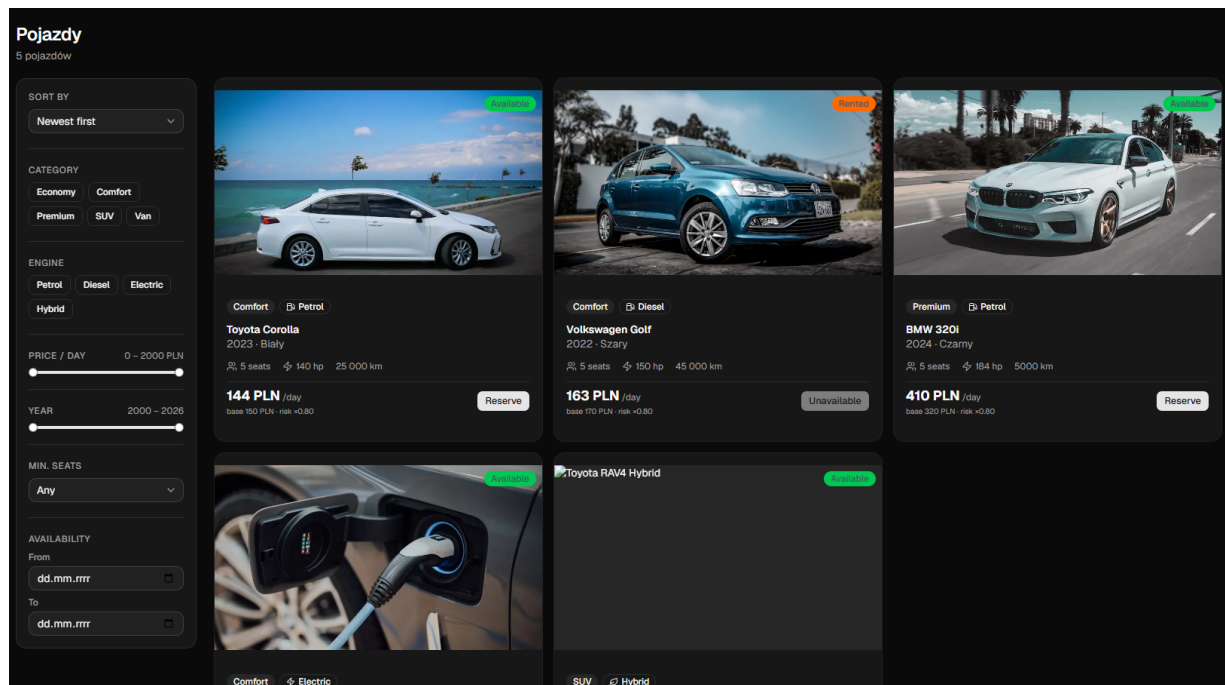
Model pojazdu przechowuje m.in. typ silnika (`EngineType`), specyfikację i zdenormalizowane agregaty ocen (`avg_rating`, `ratings_count`), aby katalog mógł sortować po ocenie bez odpytywania MongoDB przy każdym żądaniu. Unikalność VIN i tablicy realizuje **częściowy indeks unikalny** `WHERE is_active = true` — soft-delete nie blokuje ponownego wprowadzenia tego samego egzemplarza fizycznego.

Listing 2. Typ silnika i kluczowe pola pojazdu (app/models/vehicle.py)

```
class EngineType(enum.StrEnum):      # typ silnika - przechowywany, lecz nie wpływa już
    ↪ na cenę
    PETROL = "petrol"
    DIESEL = "diesel"
    ELECTRIC = "electric"
    HYBRID = "hybrid"

class Vehicle(Base):
    __tablename__ = "vehicles"
    brand: Mapped[str] = mapped_column(String(100), index=True)
    model: Mapped[str] = mapped_column(String(100))
    engine_type: Mapped[EngineType] = mapped_column(Enum(EngineType,
    ↪ native_enum=False), index=True)
    daily_base_price: Mapped[Decimal] = mapped_column(Numeric(10, 2), index=True) #
    ↪ cena bazowa za dobę
    status: Mapped[VehicleStatus] = mapped_column(..., default=VehicleStatus.AVAILABLE,
    ↪ index=True)
    avg_rating: Mapped[Decimal | None] = mapped_column(Numeric(3, 2), nullable=True)
    ↪ # zdenormalizowana ocena
    ratings_count: Mapped[int] = mapped_column(Integer, default=0, server_default="0")
    ↪ # liczba ocen (szybki katalog)
    category_id: Mapped[uuid.UUID] = mapped_column(Uuid, ForeignKey("categories.id"),
    ↪ index=True) # FK → kategoria cenowa
```

Rysunek 4. Katalog pojazdów z filtrami, oceną i ceną dzienną (odpowiada Listingom 1–2).

**8.2. Uwierzytelnianie i bezpieczeństwo**

Hasła hashowane są algorytmem bcrypt; tokeny JWT podpisywane są kluczem z konfiguracji, a w ładunku niosą identyfikator użytkownika, typ tokenu (access/refresh), czas wygaśnięcia i rolę.

Listing 3. Generowanie tokenów JWT (app/core/security.py)

```

def _create_token(subject, token_type, expires_delta, extra=None) -> str:
    expire = datetime.now(UTC) + expires_delta # moment
    ↪ wygaśnięcia tokenu
    payload = {"sub": subject, "exp": expire, "type": token_type} # sub = id usera,
    ↪ type = access/refresh
    if extra:
        payload.update(extra) # dodatkowe pola,
    ↪ np. rola
    # podpis kluczem serwera (HS256) - bez klucza nie da się sfałszować tokenu
    return jwt.encode(payload, settings.SECRET_KEY, algorithm=settings.JWT_ALGORITHM)

def create_access_token(subject: str, role: UserRole) -> str:
    return _create_token(subject, "access",
        timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES), #
    ↪ krótki TTL (np. 30 min)
        {"role": role.value}) #
    ↪ rola w ładunku tokenu

```

Logowanie weryfikuje aktywność i potwierdzenie konta, a hashowanie/weryfikację hasła (operacje CPU-bound) wykonuje w puli wątków, by nie blokować pętli asynchronicznej.

Listing 4. Logowanie i wystawianie tokenów (app/services/auth_service.py)

```

async def login_user(body: LoginRequest, db: AsyncSession) -> tuple[TokenResponse,
    ↪ User]:
    user = await user_repository.get_by_email(db, body.email)
    if user is None:
        raise InvalidCredentialsError("User not found")
    if not user.is_active:
        raise InvalidCredentialsError("Account is disabled") # konto
        ↪ zablokowane
    if not user.is_verified:
        raise InvalidCredentialsError("Email address is not verified") # wymagane
        ↪ potwierdzenie e-mail

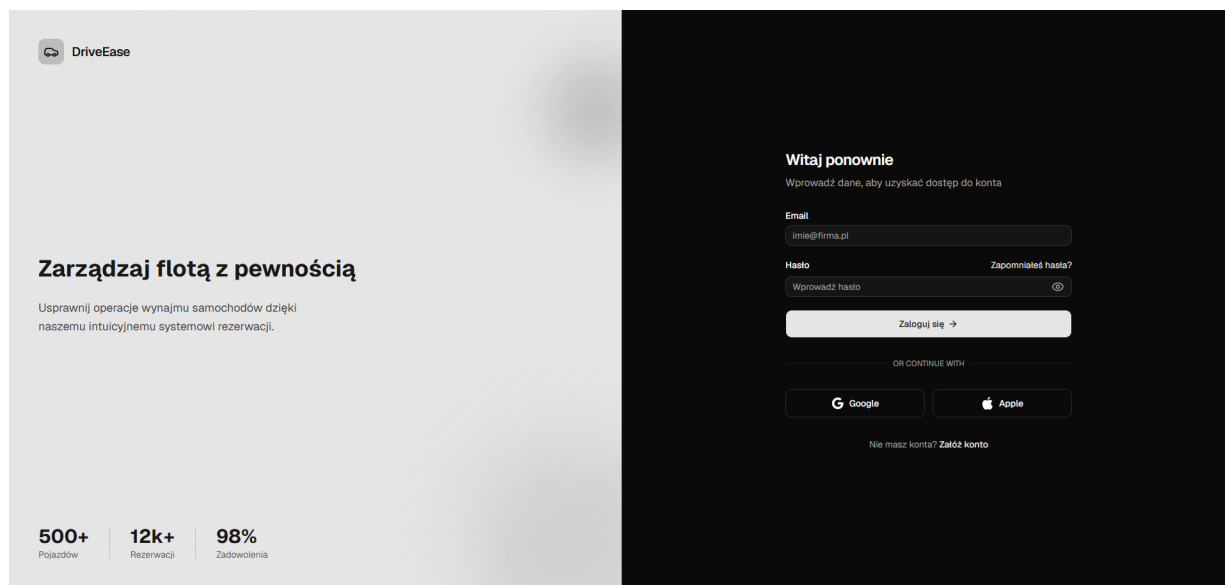
    loop = get_running_loop()
    # bcrypt jest CPU-bound → liczymy w puli wątków, by nie blokować pętli asyncio
    is_valid = await loop.run_in_executor(None, verify_password, body.password,
    ↪ user.hashed_password)
    if not is_valid:
        raise InvalidCredentialsError("Wrong password")

    access_token = create_access_token(subject=str(user.id), role=user.role) #
    ↪ krótki token dostępu
    refresh_token = create_refresh_token(subject=str(user.id), role=user.role) # długi
    ↪ token odświeżający
    await user_repository.update_last_login(db, user, datetime.now(tz=UTC)) #
    ↪ zapis ostatniego logowania
    return TokenResponse(access_token=access_token, refresh_token=refresh_token), user

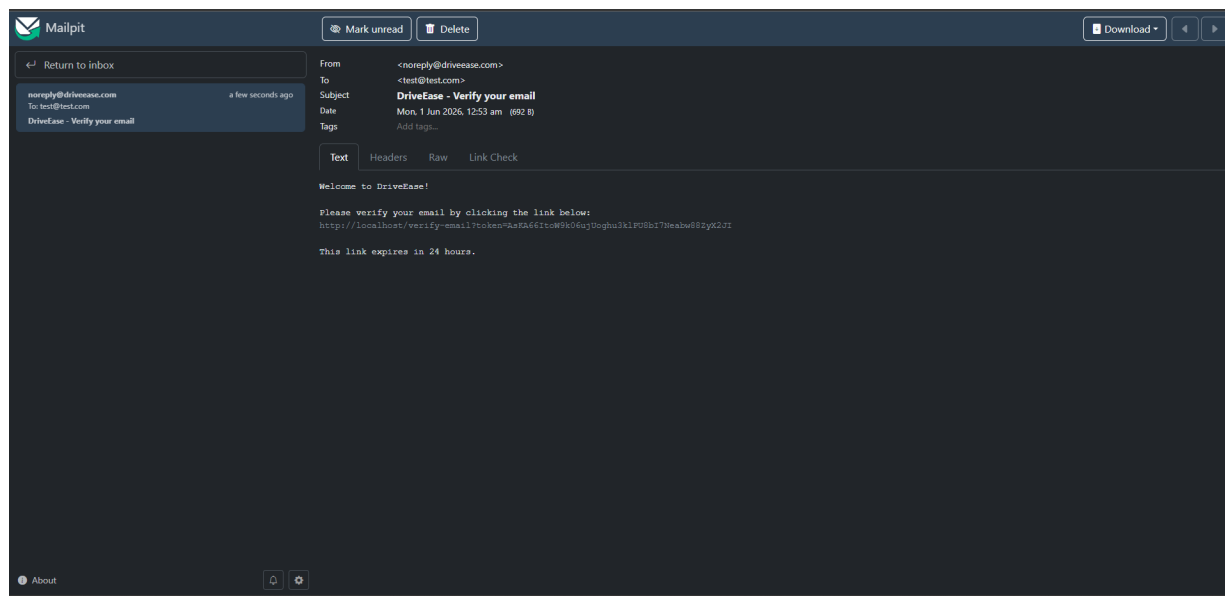
```

Odświeżanie tokenów stosuje **rotację z natychmiastowym unieważnieniem** — stary refresh-token trafia na blacklistę w Redisie, więc nie da się go użyć ponownie nawet po wycieku. Reset hasła jest chroniony przed enumeracją kont (zawsze HTTP 200) i 60-sekundowym throttlingiem.

Rysunek 5. Ekran logowania DriveEase (odpowiada Listingowi 4).



Rysunek 6. Skrzynka Mailpit z mailem weryfikacyjnym po rejestracji.



Autoryzacja po roli realizowana jest deklaratywnie przez zależność `require_roles`, a tożsamość użytkownika rozwiązywana jest z tokenu — najpierw z **cache w Redisie**, dopiero potem z bazy.

Listing 5. Zależność autoryzująca po roli (`app/core/deps.py`)

```
def require_roles(*allowed_roles: UserRole) -> Callable[..., Awaitable[User]]:
    async def _check_role(current_user: CurrentUser) -> User: # CurrentUser = user
        ↪ wyciągnięty z tokenu
        if current_user.role not in allowed_roles: # rola spoza
            ↪ dozwolonych?
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
                                detail="Insufficient permissions") # → 403 Forbidden
        return current_user
    return _check_role # zwracamy zależność wpinaną w endpoint przez
    ↪ Depends(require_roles(...))
```


8.3. Dynamiczna wycena i scoring ryzyka

Cena wynajmu liczona jest według wzoru $\text{base_price} \cdot \text{category_multiplier} \cdot \text{risk_factor} \cdot \text{dni}$. Mnożnik ryzyka odwzorowuje `risk_score` klienta (0–100) na przedział [0.8, 1.5] — klient z czystą historią dostaje zniżkę, klient ryzykowny płaci więcej.

Listing 6. Mapowanie ryzyka na mnożnik ceny (`app/services/rental_service.py`)

```
def compute_risk_multiplier(user_risk_score: Decimal | None) -> Decimal:
    """Score 0..100 → mnożnik ceny w [0.8, 1.5]."""
    if user_risk_score is None:
        return Decimal("1.0000")
    if user_risk_score < Decimal("20"):
        return Decimal("0.8000") # -20% (lojalność / czysta historia)
    if user_risk_score < Decimal("40"):
        return Decimal("0.9000") # -10%
    if user_risk_score < Decimal("60"):
        return Decimal("1.0000") # neutralnie
    if user_risk_score < Decimal("80"):
        return Decimal("1.2000") # +20%
    return Decimal("1.5000") # +50%
```

Endpoint wyceny zwraca pełne **rozbicie ceny** (kwota bazowa, korekta za ryzyko, suma), aby UI mógł je czytelnie pokazać. Cała arytmetyka pieniężna używa typu `Decimal` (brak błędów float).

Listing 7. Wyliczenie oferty cenowej (`app/services/pricing_service.py`)

```
days = _days_between(start_date, end_date) # liczba dni (min. 1)
daily_base = vehicle.daily_base_price # cena bazowa pojazdu za dobę
category_mult = vehicle.category.price_multiplier # mnożnik kategorii (premium >
    → economy)
risk_mult = compute_risk_multiplier(current_user.risk_score) # mnożnik z profilu
    → ryzyka klienta

base_subtotal = (daily_base * category_mult * Decimal(days)).quantize(MONEY_QUANT) #
    → cena BEZ ryzyka
total = (base_subtotal * risk_mult).quantize(MONEY_QUANT) #
    → cena Z ryzykiem
risk_adjustment = (total - base_subtotal).quantize(MONEY_QUANT) # ile dokłada/odejmuje
    → ryzyko (dla UI)
```

Rysunek 7. Podsumowanie ceny w kreatorze rezerwacji — kwota bazowa i korekta za ryzyko (odpowiada Listingom 6–7).

New Booking
Reserve a vehicle in a few steps

1 Choose vehicle — 2 **Price summary** — 3 Contact — 4 Confirm

Price summary
Review your booking details before continuing.

Hyundai Kona Electric (2023)
Comfort · 5 Seats

Period: 12 Mar 2026 – 15 Mar 2026
Duration: 3 days

Base price	250.00 PLN / day
Category multiplier	×1.20
Days	3
Subtotal	900.00 PLN
Loyalty discount (×0.80)	-180.00 PLN
Total	720.00 PLN

240.00 PLN / day · risk factor derived from your rental history.

< Back Next >

Sam `risk_score` jest **sterowany zdarzeniami** — przeliczany po każdym zwrocie pojazdu. Wynik zależy od wagi incydentów (drobny/poważny/krytyczny), **wykładniczego zaniku znaczenia w czasie** (okres półtrwania 365 dni) oraz liczby zakończonych wynajmów; klient-„stały bywalec” bez incydentów otrzymuje zniżkę lojalnościową.

Listing 8. Rdzeń silnika oceny ryzyka (`app/services/risk_scoring.py`)

```
weighted = Decimal("0")
for incident in incidents:
    age_days = (current_time - incident.created_at).days
    weight = SEVERITY_WEIGHT[incident.severity]
    weighted += weight * _recency_factor(age_days)           # 0.5 ** (age/365)

divisor = Decimal(max(completed_rentals, 1))
rate = weighted / divisor
score = NEUTRAL_BASELINE + weighted + rate * RATE_BOOST_MULTIPLIER

# Zniżka lojalnościowa: ~czysta historia + min. 5 zakończonych wynajmów
if weighted < LOYALTY_WEIGHTED_EPSILON and completed_rentals >=
    ↳ LOYALTY_RENTAL_THRESHOLD:
    score -= min(Decimal(completed_rentals) * Decimal("2"), LOYALTY_MAX_DISCOUNT)

score = max(Decimal("0"), min(Decimal("100"), score)).quantize(Decimal("0.01"))
```

Po zwrocie pojazdu wynik zapisywany jest i **unieważniany w cache Redisa**, aby kolejna wycena od razu uwzględniała nowy profil ryzyka.

Rysunek 8. Karta klienta w panelu pracownika — `risk_score`, statystyki i lista incydentów (odpowiada Listingowi 8).

Anna Nowak
anna.nowak@example.com

Profile

AN Anna Nowak
Customer

anna.nowak@example.com
+48601200300
Member since 25 May 2026

Verified Risk score: 100.0

Activity overview

Total rentals: 5
Completed: 2
Cancelled: 0
Total spent: 3760 PLN
Incidents: 2

Rental history

Rental	Period	Status	Price
Volkswagen Golf (2022) KR 67890	22 May 2026 – —	Active	1071 PLN
Toyota RAV4 Hybrid (2023) PO 22222	26 Mar 2026 – 31 Mar 2026	Completed	1260 PLN
Toyota Corolla (2023) WA 12345	25 Jan 2026 – 28 Jan 2026	Completed	405 PLN

Incidents

+ Add Incident

Incident	Category	Amount
adfta	Major	123.00 PLN
123	Major, Complaint	123.00 PLN

Internal notes

+ Add note

No notes yet.

8.4. Wynajem: odbiór, zwrot i historia

Pracownik potwierdza odbiór (powstaje rekord `Rental` ze stanem licznika i paliwa), a przy zwrocie liczona jest cena finalna, zapisywane jest jej rozbiecie, aktualizowany jest `risk_score` klienta, a zdarzenie trafia do logu w MongoDB.

Listing 9. Finalizacja zwrotu i przeliczenie ryzyka (`app/services/rental_service.py`)

```
base_price = (reservation.total_price + body.extra_charges).quantize(Decimal("0.01"))
↳ # cena + dopłaty (szkody itp.)
customer = await user_repository.get_by_id(db, reservation.user_id)
risk_multiplier = compute_risk_multiplier(customer.risk_score if customer else None)
↳ # mnożnik ryzyka klienta
final_price = (base_price * risk_multiplier).quantize(Decimal("0.01"))
↳ # finalna kwota do zapłaty

breakdown = await rental_repository.create_price_breakdown(      # zapis składników
↳ ceny (audyt / transparentność)
    db, rental_id=rental.id, base_price=base_price,
    risk_multiplier=risk_multiplier, final_price=final_price)

await reservation_repository.update_status(db, reservation,
↳ ReservationStatus.COMPLETED) # rezerwacja zakończona
# Event-driven: przelicz risk_score z całej historii + unieważnij cache (redis)
await risk_scoring.recompute_and_persist(db, reservation.user_id, get_redis())
```

Rysunek 9. Formularz zwrotu pojazdu (stan licznika, poziom paliwa, dopłaty) (odpowiada Listingowi 9).

8.5. System recenzji

Recenzję może wystawić wyłącznie klient, który **zakończył** dany wynajem; duplikaty blokuje indeks unikalny w MongoDB. Po każdej operacji przeliczane są agregaty oceny pojazdu i zapisywane w PostgreSQL (rekord pojazdu blokowany `SELECT ... FOR UPDATE`, by zserializować równoległe zapisy).

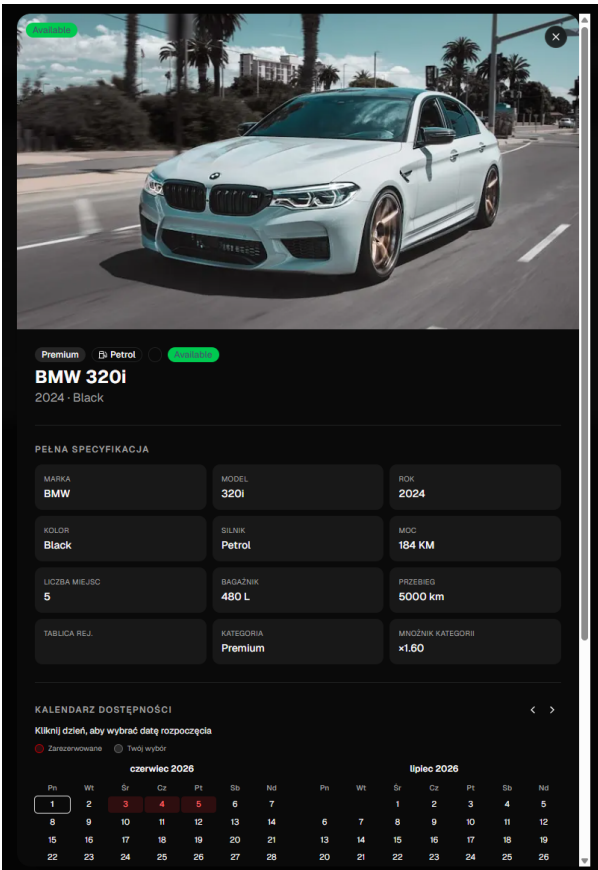
Listing 10. Tworzenie recenzji i odświeżenie agregatu (`app/services/review_service.py`)

```
if reservation.user_id != current_user.id:
    raise HTTPException(403, "You can only review your own rentals")    # tylko własny
    ↪ wynajem
if reservation.status != ReservationStatus.COMPLETED:
    raise HTTPException(422, "You can only review a completed rental")  # tylko
    ↪ zakończony wynajem

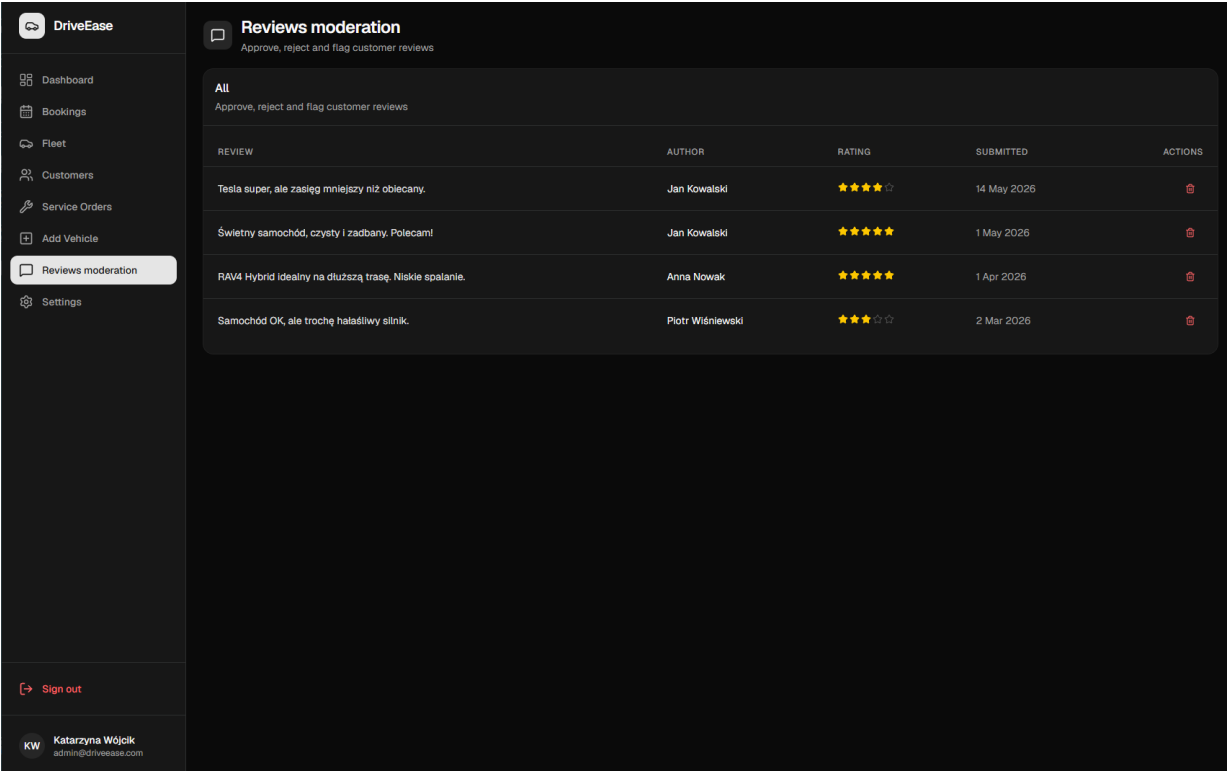
try:
    doc = await review_repository.insert(mongo, user_id=current_user.id,    # zapis
    ↪ recenzji do MongoDB
        vehicle_id=reservation.vehicle_id, rental_id=rental.id,
        rating=body.rating, comment=body.comment, ...)
except DuplicateKeyError:
    # unikalny
    ↪ indeks (rental_id, user_id)
    raise HTTPException(409, "A review already exists for this rental")  # → 409:
    ↪ jedna opinia na wynajem

await _refresh_vehicle_rating(db, mongo, reservation.vehicle_id) # przelicz avg +
↪ count i zapisz do Postgresa
```

Rysunek 10. Sekcja recenzji pojazdu i formularz wystawienia opinii (odpowiada Listingowi 10).



Rysunek 11. Moderacja recenzji w panelu administratora (odpowiada Listingowi 10).



8.6. Maile transakcyjne

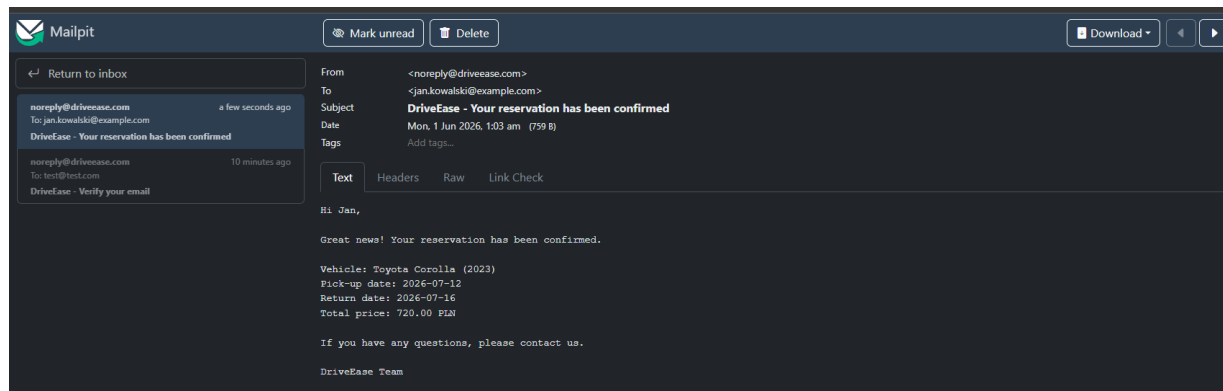
Wiadomości (weryfikacja konta, reset hasła, potwierdzenie rezerwacji) budowane są jako Email-Message i wysyłane przez SMTP; w środowisku deweloperskim przechwytuje je Mailpit. Błędy

wysyłki są logowane, ale nie przerywają obsługi żądania.

Listing 11. Mail potwierdzający rezerwację (app/core/email.py)

```
def send_reservation_confirmed_email(to_email, first_name, vehicle_name,
                                    start_date, end_date, total_price):
    msg = EmailMessage()
    msg["Subject"] = "DriveEase - Your reservation has been confirmed" # temat
    ↪ wiadomości
    msg["From"] = settings.SMTP_FROM_EMAIL
    msg["To"] = to_email
    msg.set_content(
        ↪ tekstowa maila
        f"Hi {first_name},\n\nGreat news! Your reservation has been confirmed.\n\n"
        f"Vehicle: {vehicle_name}\nPick-up date: {start_date}\nReturn date:
        ↪ {end_date}\n"
        f"Total price: {total_price} PLN\n\nDriveEase Team")
    _send_email(msg) # wysyłka SMTP; ewentualny błąd jest logowany, ale nie przerywa
    ↪ żądania
```

Rysunek 12. Mail z potwierdzeniem rezerwacji w Mailpit (odpowiada Listingowi 11).



9. Implementacja — frontend

Frontend to aplikacja **Next.js (App Router)** z TypeScriptem. Komunikacja z backendem odbywa się przez warstwę **hooków SWR** (src/hooks/), stan sesji i ustawień trzymany jest w **kontekstach React**, a logika domenowa niezależna od UI wydzielona jest do **czystych funkcji** w **src/lib/** (objętych testami jednostkowymi). Dzięki temu komponenty pozostają cienkie, a testowalna logika — odseparowana.

9.1. Ochrona tras (middleware)

Dostęp do tras chroniony jest **przed renderowaniem** — middleware sprawdza obecność ciasteczka sesji i przekierowuje: niezalogowanego z trasy chronionej na logowanie, a zalogowanego z logowania/rejestracji na pulpit.

Listing 12. Middleware ochrony tras (src/proxy.ts)

```
const PUBLIC_PATHS = ['/', '/register', '/forgot-password', '/reset-password',
  ↪ '/verify-email']; // trasy bez logowania

export function proxy(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const accessToken = request.cookies.get('access_token')?.value; // sesja =
  ↪ ciasteczko httpOnly
  const isPublicPath = PUBLIC_PATHS.includes(pathname);

  // Zalogowany na stronie logowania → pulpit
  if (isPublicPath && accessToken) {
    return NextResponse.redirect(new URL('/dashboard', request.url));
  }
  // Niezalogowany na trasie chronionej → logowanie
  if (!isPublicPath && !accessToken) {
    return NextResponse.redirect(new URL('/', request.url));
  }
  return NextResponse.next(); // dostęp dozwolony - renderuj stronę
}
```

9.2. Kontekst sesji (AuthContext)

Sesja użytkownika udostępniana jest globalnie przez AuthProvider, który wystawia akcje login / register / logout / refreshUser. Operacje korzystają z ciasteczek httpOnly (credentials: 'include'), a dane z API są mapowane na model UI (mapUserFromApi).

Listing 13. Akcje logowania i pobrania sesji (src/contexts/AuthContext.tsx)

```
const refreshUser = useCallback(async () => {
  try {
    const res = await fetch('/api/users/me', { credentials: 'include' }); // pobierz
    ↪ profil po ciasteczku sesji
    if (res.ok) setUser(mapUserFromApi(await res.json())); // mapowanie odpowiedzi
    ↪ API → model UI
    else setUser(null); // brak / nieważna sesja
  } catch {
    setUser(null);
  } finally {
    setIsLoading(false); // koniec ładowania (pokaż UI)
  }
}, []);

KeywordTokconst login = useCallback(async (email: string, password: string) => {
  const res = await fetch('/api/auth/login', {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    credentials: 'include', body: JSON.stringify({ email, password }), // backend
    ↪ ustawia ciasteczka sesji
  });
  if (!res.ok) throw new Error((await res.json()).detail ?? 'Login failed'); //
  ↪ komunikat błędu z API
  setUser(mapUserFromApi(await res.json()));
  await refreshUser(); // domknij stan sesji
}, [refreshUser]);
```

9.3. Pobieranie danych (SWR) — wycena spięta z backendem

Hooki `use*` opakowują SWR. Hook wyceny buduje klucz zapytania tylko gdy komplet parametrów jest dostępny (warunkowe `key = ... : null` wstrzymuje zapytanie), a wynik z endpointu `/pricing/quote` trafia bezpośrednio do komponentu podsumowania ceny — to ten sam mechanizm, który po stronie backendu opisuje Listing 7.

Listing 14. Hook wyceny oparty o SWR (`src/hooks/usePriceQuote.ts`)

```
export function usePriceQuote(vehicleId: string | null, startDate: string, endDate:
  ↳ string) {
  const enabled = !!vehicleId && !!startDate && !!endDate && startDate !== endDate; //
  ↳ komplet parametrów?
  const key = enabled
    ? `/api/pricing/quote?vehicle_id=${vehicleId}&start_date=${startDate}&end_date=${
      ↳ endDate}`
    : null; // null → SWR nie wysyła zapytania

  const { data, isLoading, error } = useSWR<PriceBreakdown>(key, fetcher); // cache +
  ↳ automatyczna rewalidacja
  return { quote: data ?? null, isLoading: enabled && isLoading, error }; // gotowy
  ↳ wynik dla komponentu
}
```

9.4. Mutacje i spójność cache

Operacje zapisu (np. dodanie recenzji) po sukcesie **rewalidują** odpowiednie klucze SWR, dzięki czemu listy odświeżają się automatycznie bez ręcznego przeładowania. Funkcja `revalidateReviewLists` unieważnia wszystkie klucze związane z recenzjami (listy per-pojazd, moderację, „wynajmy do oceny”).

Listing 15. Mutacja recenzji z inwalidacją cache (`src/hooks/useReviewMutations.ts`)

```
function revalidateReviewLists(): void {
  globalMutate((key) => // unieważnij wszystkie
    ↳ pasujące klucze SWR
    typeof key === 'string' &&
    ((key.startsWith('/api/vehicles/') && key.includes('/reviews')) || // listy
    ↳ recenzji per pojazd
    key.startsWith('/api/reviews') || // moderacja
    ↳ (admin)
    key.startsWith('/api/users/me/rentals'))); // "wynajmy do
    ↳ oceny"
}

const submit = useCallback(async (payload: CreateReviewPayload) => {
  const res = await fetch('/api/reviews', {
    method: 'POST', credentials: 'include',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ rental_id: payload.rentalId, rating: payload.rating,
      comment: payload.comment });
  });
  if (!res.ok) throw new Error(await readError(res)); // wyciągnij detal błędu z
  ↳ odpowiedzi API
  revalidateReviewLists(); // odśwież listy po zapisie (bez ręcznego
  ↳ reloadu)
  return mapReview(await res.json());
}, []);
```


9.5. Motyw jasny/ciemny i wielojęzyczność

Motyw i język trzymane są w kontekście i utrwalane w `localStorage`; zmiana motywu przełącza klasę `dark` na elemencie `html` (Tailwind), a język wybiera odpowiedni słownik tłumaczeń.

Listing 16. Kontekst ustawień: motyw + język (`src/contexts/SettingsContext.tsx`)

```
export type Theme = 'light' | 'dark';
export type Language = 'en' | 'pl';

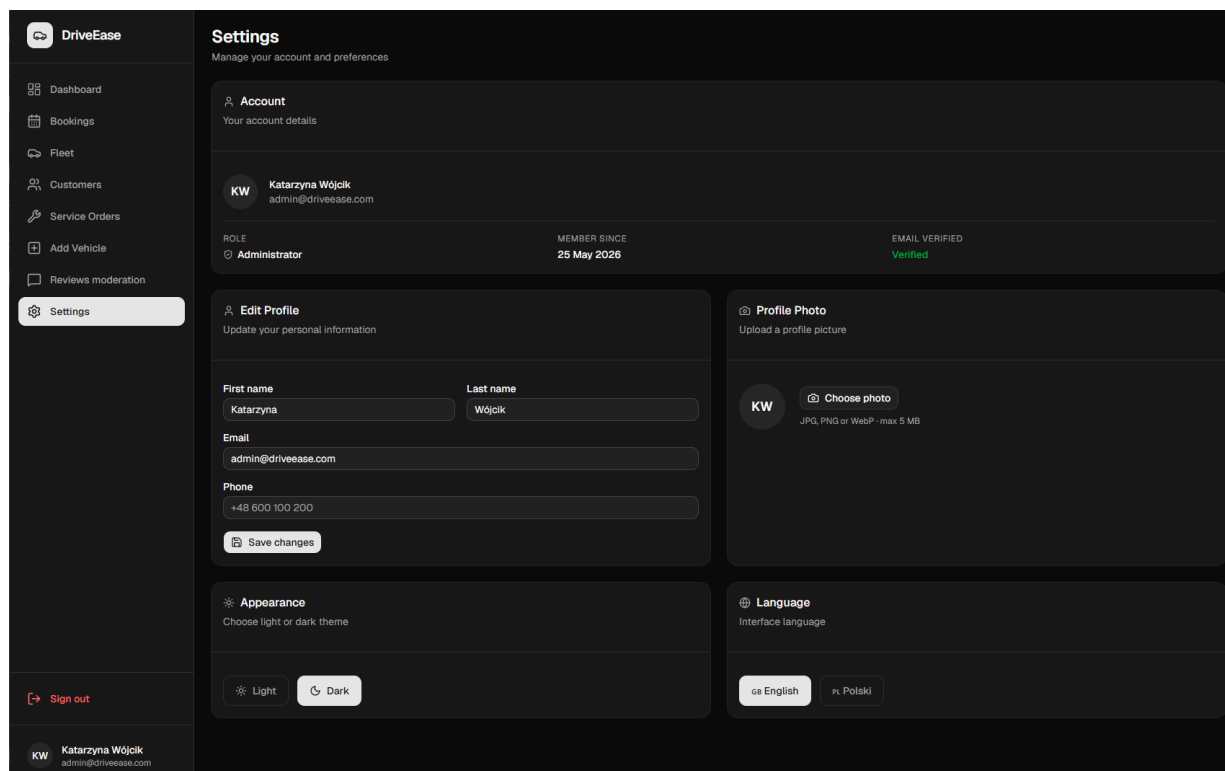
const setTheme = (t: Theme) => {
  setThemeState(t);
  localStorage.setItem('theme', t); // utrwal wybór
  ↪ między sesjami
  document.documentElement.classList.toggle('dark', t === 'dark'); // przełącz klasę
  ↪ .dark (Tailwind)
};
const setLanguage = (l: Language) => {
  setLanguageState(l);
  localStorage.setItem('language', l); // zapamiętaj język interfejsu
};
```

Tłumaczenia rozwiązuje hook `useTranslation`, który wybiera słownik wg aktualnego języka i wspiera interpolację zmiennych.

Listing 17. Hook tłumaczeń (`src/i18n/useTranslation.ts`)

```
export function useTranslation() {
  const { language } = useSettings(); // aktualny język z kontekstu ustawień
  const dict = translations[language]; // wybór słownika PL / EN
  const t = (key: TranslationKey, vars?: Record<string, string | number>): string => {
    const value = dict[key] ?? key; // brak tłumaczenia → pokaż klucz
    ↪ (bezpieczny fallback)
    return vars ? tFormat(value, vars) : value; // interpolacja zmiennych, np. {count}
  };
  return { t, language };
}
```

Rysunek 13. Ustawienia konta w trybie ciemnym i języku polskim (odpowiada Listingom 16–17).



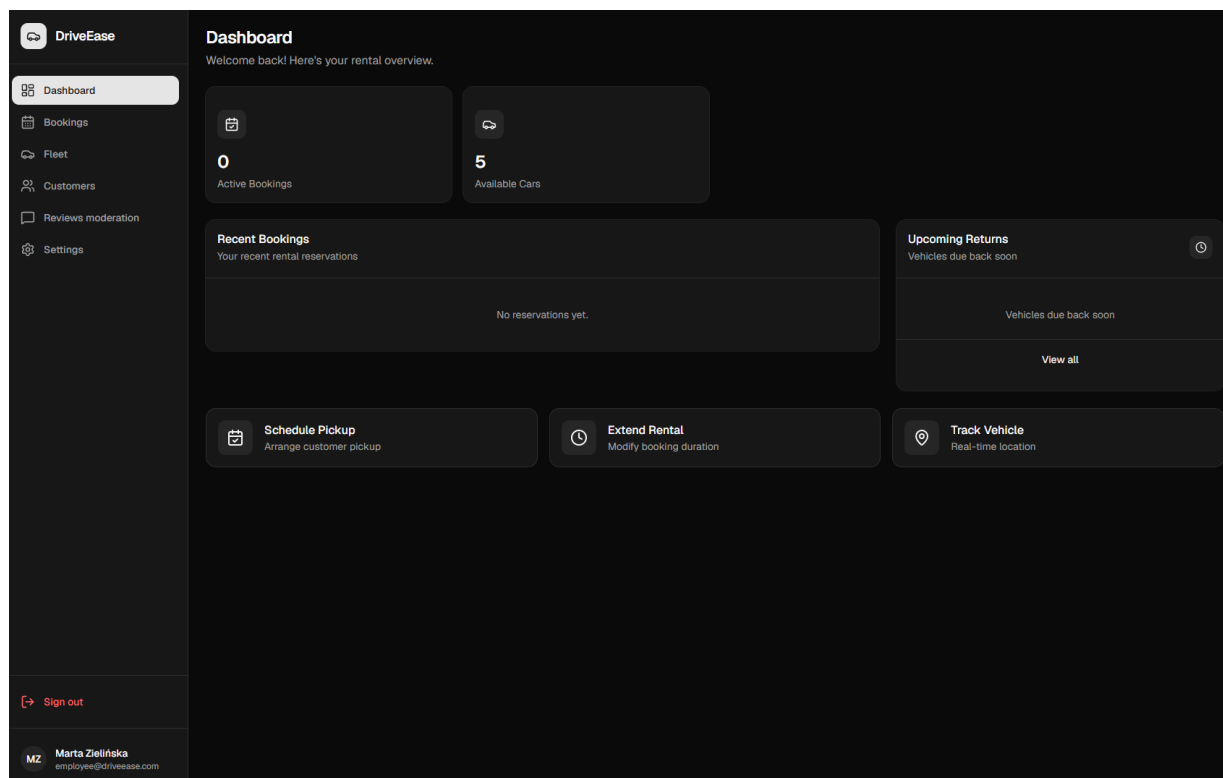
9.6. Panele według ról

Nawigacja boczna jest **filtrowana po roli** — klient widzi katalog i swoje rezerwacje, personel (pracownik/serwisant/admin) widzi flotę, klientów, zlecenia serwisowe i moderację.

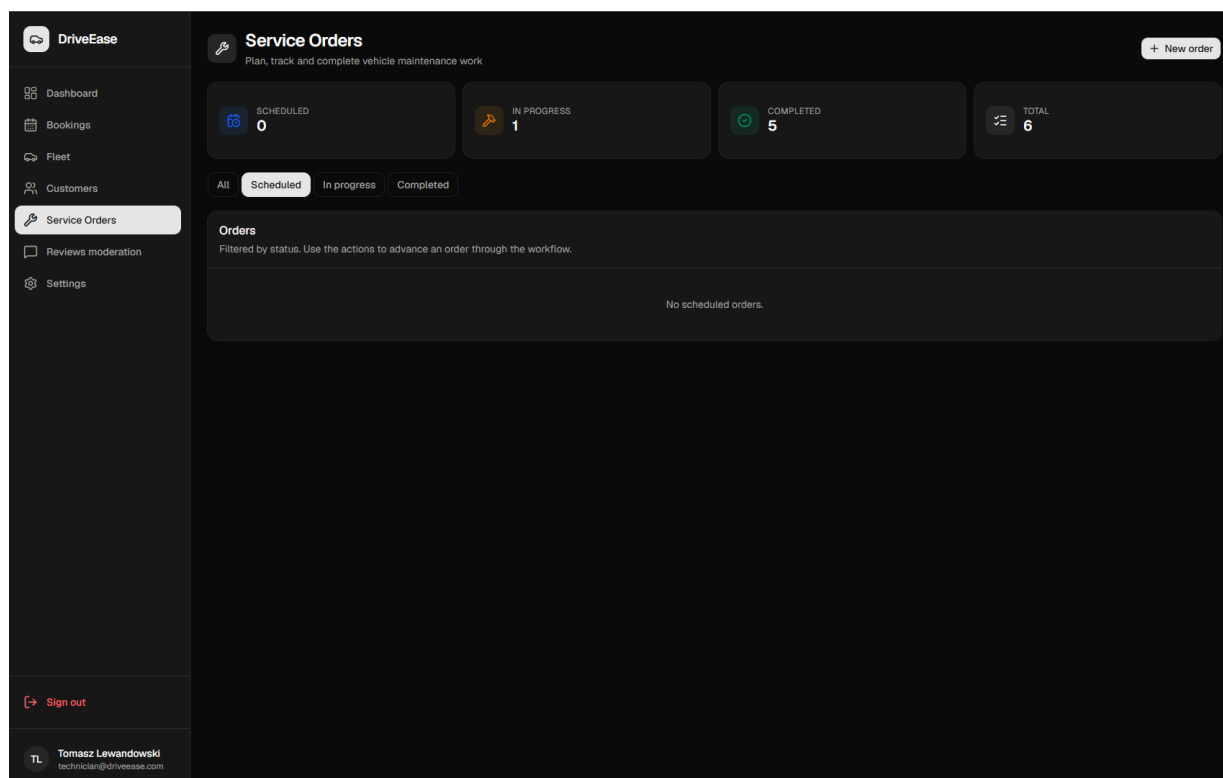
Listing 18. Filtrowanie nawigacji po roli (`src/data/dashboard/constants.ts`)

```
export function getFilteredNavigation(role?: UserRole): NavItem[] {
  const staff = isStaffRole(role); // employee / technician / admin?
  return navigation.filter((item) => {
    if (item.roles && (!role || !item.roles.includes(role))) return false; // pozycja
    ↪ dla konkretnych ról
    if (item.staffOnly && !staff) return false; // tylko dla personelu
    if (item.hideForStaff && staff) return false; // ukryj przed personelem (katalog
    ↪ klienta)
    return true;
  });
}
// "Fleet"/"Customers" → staffOnly; "Service orders" → ['technician','admin'];
// "Add vehicle" → ['admin']; "Vehicles" (katalog) → hideForStaff
```

Rysunek 14. Pulpit pracownika — statystyki, nadchodzące zwroty, tabela rezerwacji (odpowiada Listingowi 18).



Rysunek 15. Panel serwisanta — zlecenia serwisowe i statystyki (odpowiada Listingowi 18).



9.7. Logika domenowa po stronie klienta (czyste funkcje)

Reguły niezależne od UI (np. walidacja siły hasła, wykrycie domyślnych filtrów katalogu) są czystymi funkcjami — łatwymi do przetestowania jednostkowo (rozdz. 14).

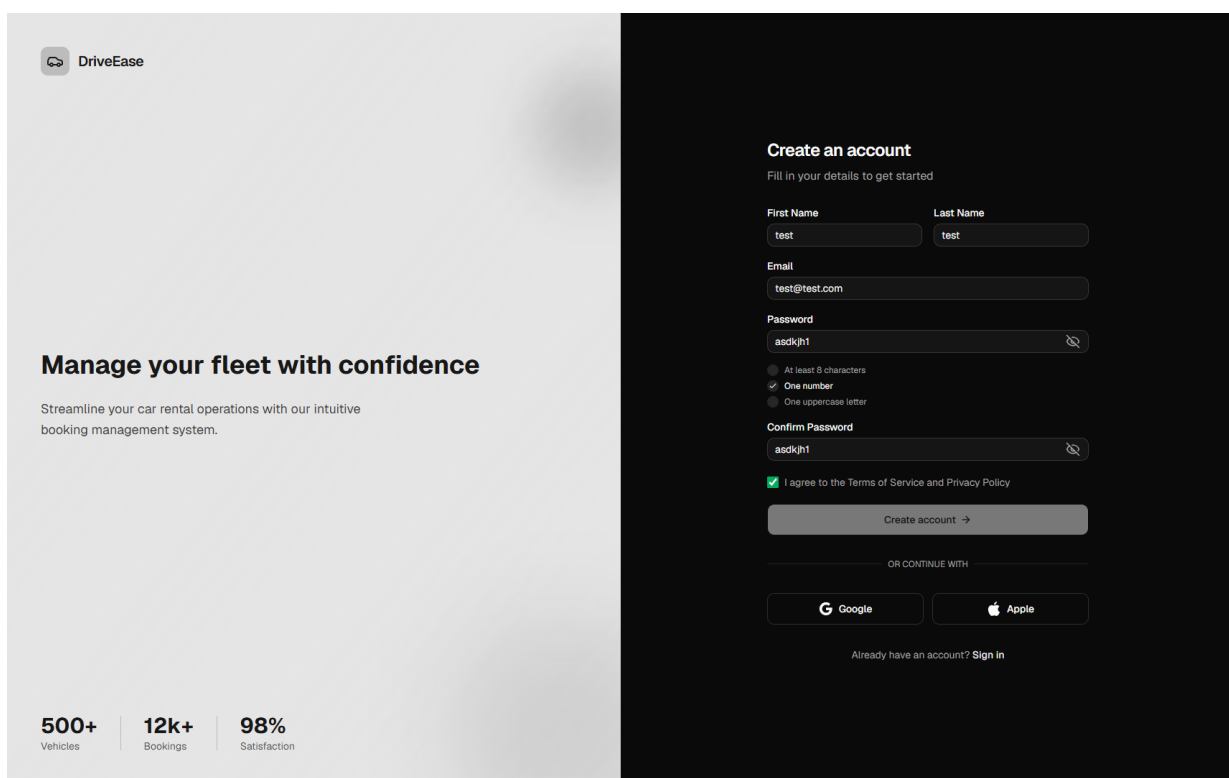
Listing 19. Walidacja siły hasła (src/lib/password.ts)

```

export function getPasswordRequirements(password: string): PasswordRequirement[] {
  return [
    { label: 'pwd.min8', met: password.length >= 8 },           // min. 8 znaków
    { label: 'pwd.number', met: /\d/.test(password) },           // co najmniej 1 cyfra
    { label: 'pwd.uppercase', met: /[A-Z]/.test(password) },     // co najmniej 1 wielka
    ↪ litera
  ];
}
export function isValidPassword(password: string): boolean {
  return getPasswordRequirements(password).every((r) => r.met); // wszystkie wymagania
  ↪ spełnione
}

```

Rysunek 16. Rejestracja z dynamicznym wskaźnikiem wymagań hasła (odpowiada Listingowi 19).



10. Katalog endpointów API

Backend wystawia REST API pod prefiksem `/api`. Interaktywna dokumentacja (Swagger UI) jest dostępna pod `http://localhost/api/docs`. Kolumna „Dostęp”: *publiczny* — bez logowania; *zalogowany* — dowolna rola; *pracownik/admin*, *technik/admin*, *admin* — wg `require_roles`.

Uwierzytelnianie — `/auth`

Metoda	Ścieżka	Dostęp	Opis
POST	<code>/auth/register</code>	publiczny	rejestracja konta + wysyłka maila weryfikacyjnego
POST	<code>/auth/login</code>	publiczny	logowanie, ustawienie ciasteczek sesji
POST	<code>/auth/refresh</code>	publiczny (cookie)	rotacja tokenów (access + refresh)

Metoda	Ścieżka	Dostęp	Opis
GET	/auth/me	zalogowany	dane bieżącego użytkownika
POST	/auth/logout	zalogowany	wylogowanie (blacklista tokenów)
GET	/auth/verify-email	publiczny	potwierdzenie e-mail tokenem
POST	/auth/forgot-password	publiczny	żądanie resetu hasła
POST	/auth/reset-password	publiczny	ustawienie nowego hasła

Profil — /users

Metoda	Ścieżka	Dostęp	Opis
GET	/users/me	zalogowany	profil
PUT	/users/me	zalogowany	edycja profilu
POST	/users/me/avatar	zalogowany	upload awatara (obraz)
GET	/users/me/rentals	zalogowany	historia wynajmów (paginacja)

Katalog — /vehicles, /categories, /pricing

Metoda	Ścieżka	Dostęp	Opis
GET	/vehicles	publiczny	katalog (filtry, sortowanie, paginacja)
GET	/vehicles/{id}	publiczny	szczegóły pojazdu
GET	/vehicles/{id}/reviews	publiczny	recenzje pojazdu (paginacja)
GET	/vehicles/{id}/availability	publiczny	dostępność (zajęte terminy)
GET	/categories	publiczny	lista kategorii
GET	/pricing/quote	zalogowany	wycena z rozbiorem (ryzyko, kategoria)

Rezerwacje i wynajmy — /reservations, /rentals

Metoda	Ścieżka	Dostęp	Opis
POST	/reservations	zalogowany	utworzenie rezerwacji
GET	/reservations	zalogowany	moje rezerwacje
PUT	/reservations/{id}/cancel	zalogowany	anulowanie (właściciel)
PUT	/reservations/{id}/confirm	pracownik/admin	potwierdzenie rezerwacji
POST	/rentals/{reservation_id}/pickup	pracownik/admin	odbiór pojazdu
POST	/rentals/{rental_id}/return	pracownik/admin	zwrot + cena finalna

Recenzje — /reviews

Metoda	Ścieżka	Dostęp	Opis
POST	/reviews	zalogowany	dodanie recenzji (tylko zakończony wynajem)
GET	/reviews/me/rentals	zalogowany	id wynajmów już zrecenzowanych
GET	/reviews	admin	wszystkie recenzje (moderacja)
DELETE	/reviews/{id}	admin	usunięcie recenzji

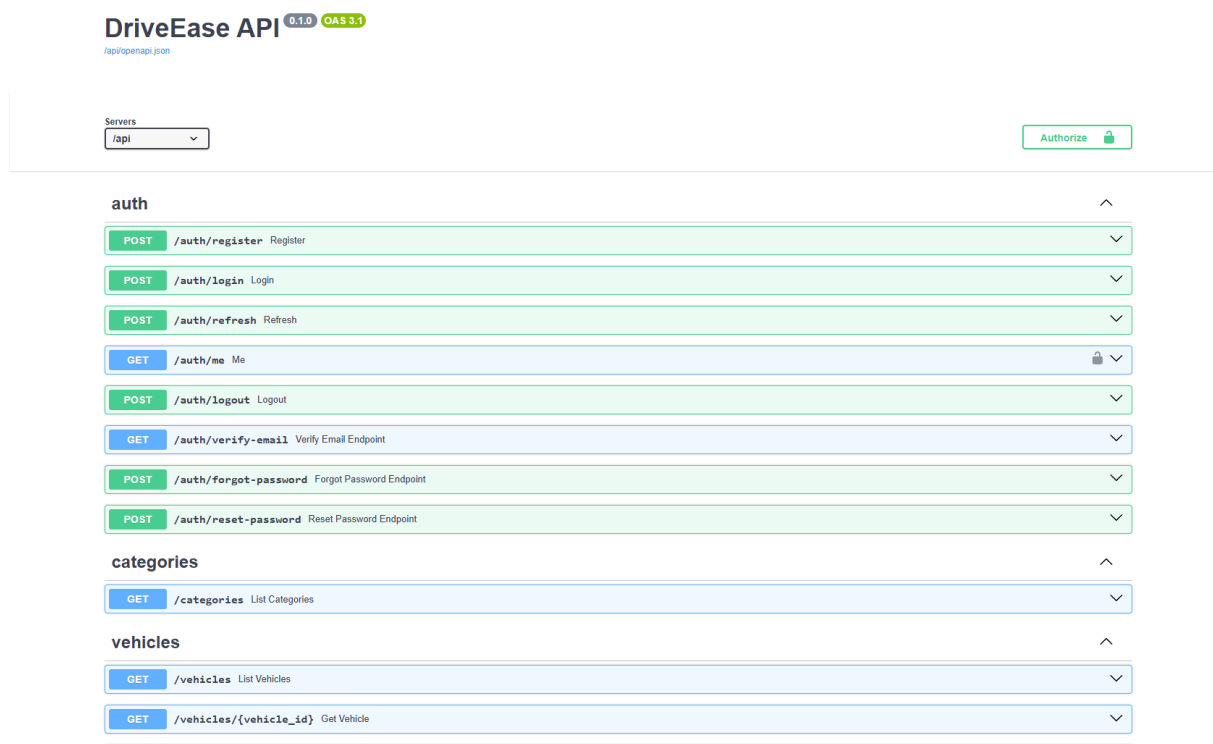
Serwis — /service-orders, /vehicles/{id}/service-orders

Metoda	Ścieżka	Dostęp	Opis
POST	/service-orders	technik/admin	utworzenie zlecenia
GET	/service-orders	technik/admin	lista zleceń
GET	/service-orders/stats	technik/admin	statystyki zleceń
GET	/service-orders/{id}	technik/admin	szczegóły zlecenia
PUT	/service-orders/{id}	technik/admin	edycja zlecenia
PUT	/service-orders/{id}/status	technik/admin	zmiana statusu
POST	/service-orders/{id}/history	technik/admin	wpis do historii serwisu
GET	/vehicles/{id}/service-orders	technik/admin	oś czasu serwisu pojazdu

Administracja — /admin, /admin/vehicles, /admin/customers

Metoda	Ścieżka	Dostęp	Opis
GET	/admin/reservations	admin	wszystkie rezerwacje
GET	/admin/users	admin	lista użytkowników
POST	/admin/vehicles	admin	dodanie pojazdu
PUT	/admin/vehicles/{id}	admin	edycja pojazdu
DELETE	/admin/vehicles/{id}	admin	usunięcie (soft-delete)
PATCH	/admin/vehicles/bulk-status	admin	masowa zmiana statusu floty
POST · DELETE	/admin/vehicles/{id}/images...	admin	zarządzanie zdjęciami pojazdu
GET	/admin/customers/{id}	pracownik/admin	szczegóły klienta
POST · DELETE	/admin/customers/{id}/incidents...	pracownik/admin	rejestr incydentów
POST · PUT · DELETE	/admin/customers/{id}/notes...	pracownik/admin	notatki o kliencie

Rysunek 17. Interaktywna dokumentacja API (Swagger UI).



11. Wzorce i decyzyje projektowe

W projekcie świadomie zastosowano kilka wzorców architektonicznych i projektowych:

- **Architektura warstwowa + wzorzec repozytorium.** Rozdzielenie router → serwis → repozytorium izoluje logikę domenową od dostępu do danych i frameworka, co ułatwia testy jednostkowe (serwisy testowane bez HTTP, z atrapami repozytoriów).
- **Wstrzykiwanie zależności (Dependency Injection).** Mechanizm Depends FastAPI dostarcza sesję bazy, bieżącego użytkownika i strażników ról; autoryzacja jest deklaratywna (`require_roles(...)`), a nie rozsiana po kodzie.
- **Cache-Aside.** Model użytkownika czytany jest najpierw z Redisa, a dopiero przy „pudle” z PostgreSQL (i wówczas zapisywany do cache). Zapisy (np. nowy `risk_score`, logowanie) **unieważniają** wpis, by uniknąć nieświeżych danych.
- **Architektura sterowana zdarzeniami (event-driven).** Zwrot pojazdu wyzwala przeliczenie `risk_score` z całej historii — efekt naliczany jest reaktywnie, a nie kopiowany ad hoc.
- **Denormalizacja z rekonsyliacją.** Agregaty ocen (`avg_rating`, `ratings_count`) trzymane są przy pojeździe w PostgreSQL (szybki odczyt katalogu), a po każdej zmianie recenzji przeliczane od źródła prawdy (MongoDB) pod blokadą wiersza.
- **Soft-delete + częściowe indeksy unikalne.** Pojazd „usuwany” flagą `is_active`; unikalność VIN/tablicy obowiązuje tylko dla aktywnych rekordów (`WHERE is_active = true`).
- **Rotacja tokenów + lista unieważnień.** Refresh-token jest jednorazowy (po użyciu trafia na blacklistę), co ogranicza skutki wycieku.
- **Persystencja poliglotyczna.** Każda baza odpowiada za to, w czym jest najlepsza: PostgreSQL — spójność transakcyjna, MongoDB — elastyczne dokumenty (logi, recenzje), Redis — ulotne dane z TTL.
- **Provider/Context + warstwa hooków (frontend).** Stan globalny (sesja, ustawienia) w kontekstach React; dostęp do danych przez hooki SWR z automatyczną rewalidacją; logika domenowa wydzielona do czystych, testowalnych funkcji.

12. Bezpieczeństwo

Bezpieczeństwo zaadresowano na kilku poziomach:

- **Hasła** — hashowane bcryptem (passlib), nigdy nie przechowywane jawnie.
- **Sesja** — JWT (access + refresh) przenoszone w ciasteczkach **httpOnly**; rotacja refresh-tokenu i blacklista w Redisie (klucz to **hash SHA-256** tokenu, nie sam token).
- **Autoryzacja** — kontrola dostępu oparta o role (**require_roles**), sprawdzana deklaratywnie na poziomie endpointu; dodatkowo „obrona w głąb” w serwisach krytycznych (np. usuwanie recenzji).
- **Reset hasła** — ochrona przed enumeracją kont (zawsze HTTP 200) i 60-sekundowy throttling; tokeny jednorazowe z TTL, kasowane atomowo (**GETDEL**).
- **Walidacja danych** — schematy Pydantic na wejściu API oraz ograniczenia **CHECK/UNIQUE/FK** na poziomie bazy (np. **risk_score** 0..100, poziom paliwa 0..100, **return_date** > **pickup_date**).
- **CORS** — ograniczony do skonfigurowanych origin-ów, z jawną listą metod i nagłówków.

13. Przepływy użytkownika

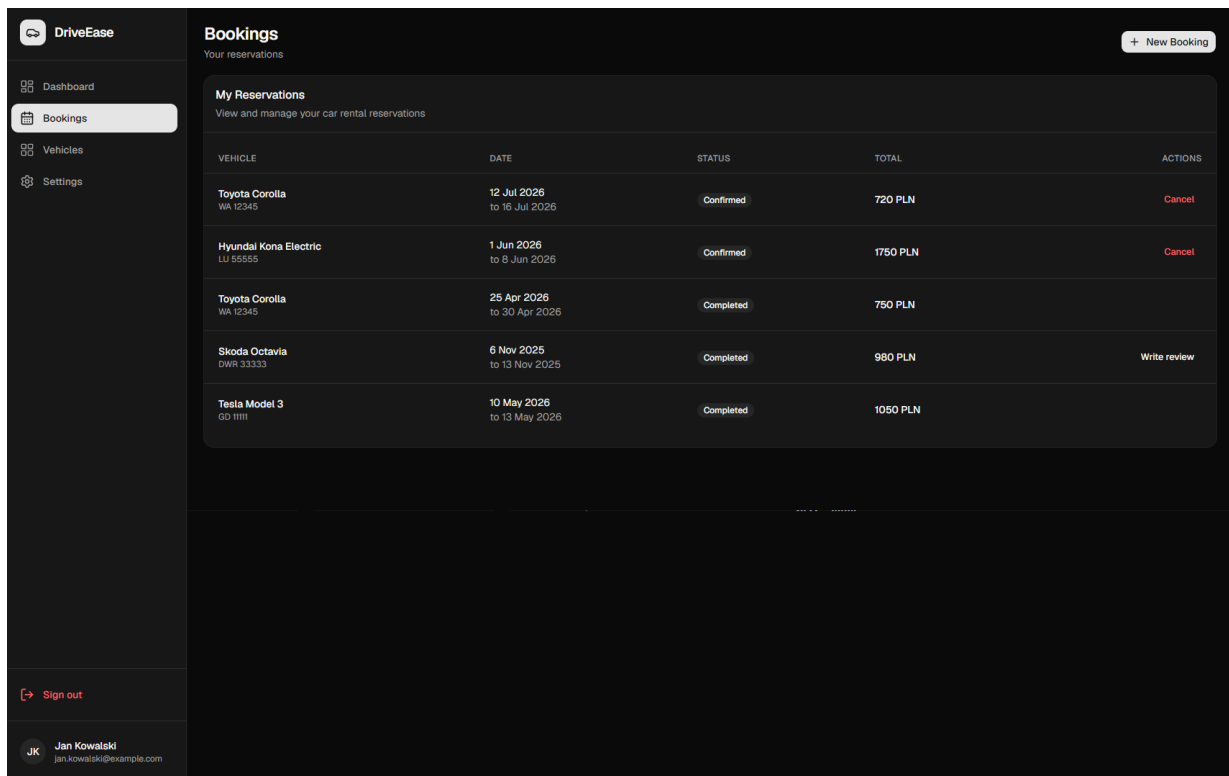
Rejestracja i aktywacja konta. Użytkownik wypełnia formularz → backend tworzy konto i zapisuje token weryfikacyjny w Redisie (TTL 24 h) → mail z linkiem trafia do Mailpit → kliknięcie linku (**/verify-email**) ustawia **is_verified = true**. Dopiero wtedy logowanie jest możliwe.

Logowanie. Formularz → **/auth/login** → walidacja, wystawienie tokenów, ustawienie ciasteczek → middleware przepuszcza na **/dashboard**, a **AuthContext** pobiera profil (**/users/me**).

Rezerwacja → odbiór → zwrot. Klient w kreatorze wybiera daty i pojazd (wycena na żywo z **/pricing/quote**) → tworzy rezerwację → pracownik potwierdza i realizuje **odbiór** (stan licznika, paliwo) → po najmie realizuje **zwrot**: liczona jest cena finalna, zapisywane rozbicie, a **risk_score** klienta jest przeliczany.

Recenzja. Po zakończonym wynajmie klient wystawia ocenę (1-5 + komentarz) → recenzja trafia do MongoDB, agregaty pojazdu są odświeżane → administrator może moderować/usunąć opinię.

Rysunek 18. Lista „moich rezerwacji” / historia wynajmów klienta.



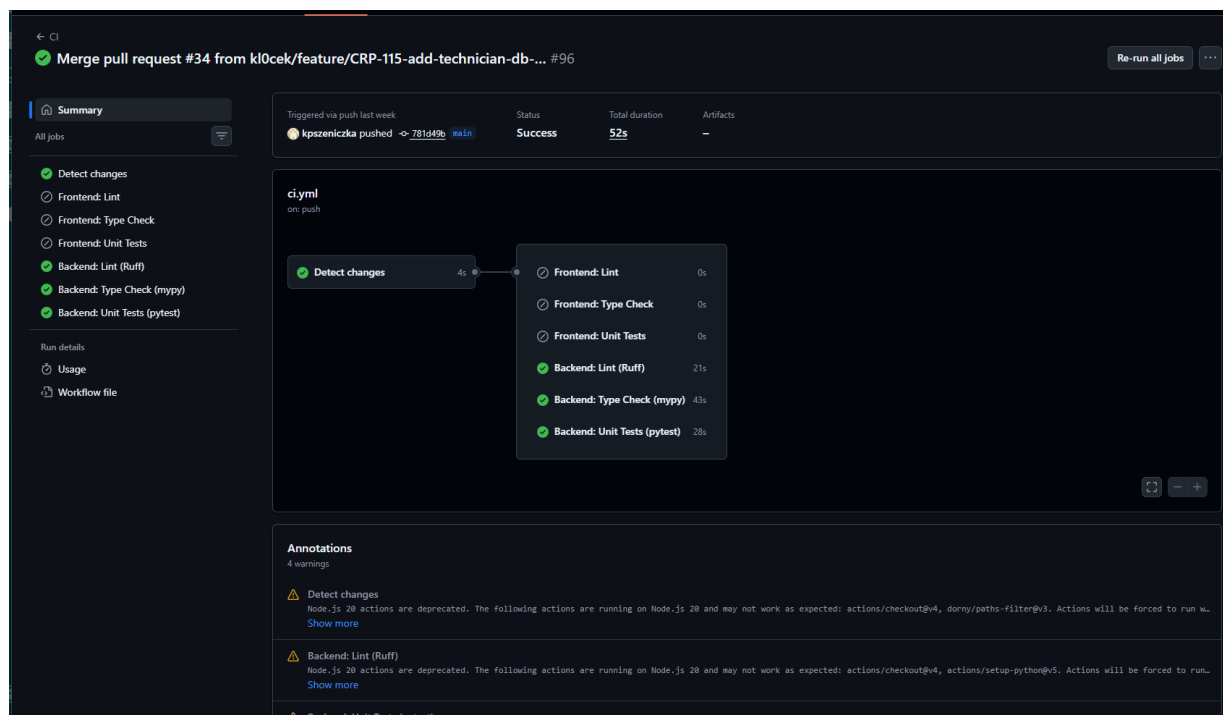
14. Testy

W projekcie zastosowano **trzy poziomy testów**, uruchamiane automatycznie w CI (GitHub Actions).

Backend pokrywają testy jednostkowe i integracyjne w `pytest` (ponad 140 przypadków, m.in. `test_auth_service`, `test_risk_scoring`, `test_risk_scoring_integration`, `test_security`, `test_token_blacklist`, `test_user_cache`, routery `auth/users/vehicles/reservations`). Sprawdzają one logikę bezpieczeństwa (hashowanie i weryfikacja haseł, dekodowanie i unieważnianie JWT), poprawność scoringu ryzyka (wagi incydentów, zanik znaczenia w czasie, zniżka lojalnościowa, zakres 0–100) oraz kontrakty endpointów (kody statusów, walidacja, autoryzacja po roli). Backend objęty jest też pomiarem pokrycia (`pytest --cov`), lintingiem (Ruff) i statyczną kontrolą typów (mypy). Testy korzystają z lekkiej bazy SQLite (wariant typu `TEXT[]` → JSON), co pozwala uruchamiać je bez pełnej infrastruktury.

Frontend testowany jest jednostkowo w `Jest` (≈23 przypadki) — logika pomocnicza w `src/lib/` (`availability`, `filters`, `formatters`, `password`, `utils`), czyli wyliczanie dostępności, filtrowanie/sortowanie katalogu i walidacja siły hasła; dodatkowo lint (ESLint) i `tsc --noEmit`. Całość spina **8 testów end-to-end** w Playwright (`e2e/`: `api-smoke`, `auth`, `rbac`), weryfikujących pełne ścieżki użytkownika oraz kontrolę dostępu opartą o role (czy klient nie wejdzie na zasoby personelu). Celem tej trójwarstwowej strategii było wczesne wychwytywanie regresji: szybkie testy jednostkowe pilnują logiki domenowej, a testy e2e — że zintegrowany system (frontend + API + bazy) faktycznie działa z perspektywy użytkownika.

Rysunek 19. Zielone statusy CI dla Pull Requesta (lint, type-check, testy FE i BE gdzie dokonano zmian).



15. Wnioski

Założone funkcjonalności udało się zrealizować w **stopniu w pełni zadowalającym**. Działa kompletny system kont (rejestracja, weryfikacja e-mail, logowanie JWT z rotacją i unieważnianiem tokenów, reset hasła), publiczny katalog pojazdów z filtrowaniem, panele wszystkich czterech ról (klient, pracownik, serwisant, administrator), pełny cykl rezerwacja → odbiór → zwrot, system recenzji z moderacją, motyw jasny/ciemny, dwujęzyczność (PL/EN) oraz maile transakcyjne. Zastosowanie **trzech baz danych** dobranych do charakteru danych (PostgreSQL — transakcje, MongoDB — logi i recenzje, Redis — cache i tokeny) okazało się trafne i dobrze ilustruje ideę persystencji poliglotycznej. Wyraźny podział na warstwy (backend) oraz na konteksty/hooks/czyste funkcje (frontend) sprawił, że kod jest czytelny i testowalny — co potwierdza obecność kilkuset testów w CI.

Największą wartość wniósł **silnik dynamicznej wyceny oparty o ocenę ryzyka klienta**. W stosunku do pierwotnego pomysłu (cena zależna od cen paliw/energii z publicznego API) świadomie zmieniono kierunek na klasyfikację ryzyka na podstawie historii wynajmów i incydentów. Rozwiązanie to jest bogatsze merytorycznie (zanik znaczenia incydentów w czasie, zniżka lojalnościowa, przeliczanie sterowane zdarzeniami z inwalidacją cache) i lepiej pokazuje współpracę warstw aplikacji. W efekcie **nie zrealizowano** integracji cenowej z zewnętrznym API paliw/energii — typ silnika jest przechowywany, ale nie wpływa na cenę przez API (prototyp dopłaty paliwowej został usunięty, migracja `drop_fuel_surcharge`). Drugim świadomym uproszczeniem są maile: zaimplementowano maile **transakcyjne** (weryfikacja, reset, potwierdzenie rezerwacji), natomiast **harmonogramowane maile przypominające** o nadchodzącym wynajmie nie zostały dodane — wymagałyby usługi cyklicznej (np. Celery/cron), co wykracza poza obecną architekturę bezstanowego API.

Możliwe usprawnienia: (1) przywrócenie komponentu cenowego zależnego od kosztów energii/paliwa jako osobnego, konfigurowalnego modyfikatora obok ryzyka; (2) zadania w tle (scheduler) na maile przypominające i okresową rekonsyliację zdenormalizowanych agregatów ocen; (3) płatności online; (4) obserwowalność (metryki, tracing). Projekt spełnił cele dydaktyczne — pokazał spójne połączenie frontendu, asynchronicznego API, wielu baz danych, konteneryzacji i automatyzacji jakości (CI, lint, typy, testy).

16. Instrukcja uruchomienia aplikacji

Wariant A — Docker Compose (zalecany)

Wymagania: zainstalowany **Docker** i **Docker Compose**.

```
# 1. Sklonuj repozytorium i wejdź do katalogu projektu
git clone https://github.com/kl0cek/car-rental-system.git
cd car-rental-system

# 2. Przygotuj zmienne środowiskowe (uzupełnij SECRET_KEY)
cp .env.example .env

# 3. Zbuduj i uruchom całe środowisko
docker compose up --build
```

Po starcie dostępne są:

Usługa	Adres
Aplikacja (frontend + API przez nginx)	http://localhost
Dokumentacja API (Swagger)	http://localhost/api/docs
Mailpit (podgląd maili)	http://localhost:8025
pgAdmin (podgląd bazy)	http://localhost:5050

Wypełnienie baz danymi przykładowymi (użytkownicy, pojazdy, rezerwacje, recenzje):

```
docker compose exec backend python -m scripts.seed
```

Konta testowe (hasło dla wszystkich: Password1):

Rola	E-mail
Administrator	admin@driveease.com
Pracownik	employee@driveease.com
Serwisant	technician@driveease.com
Klient	jan.kowalski@example.com

Wariant B — uruchomienie lokalne (dev)

Wymagania: **Node.js 20**, **Python 3.12** oraz działające instancje PostgreSQL, MongoDB i Redis (np. `docker compose up postgres mongo redis mailpit`).

```
# Backend
cd car-rental-backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
alembic upgrade head # migracje schematu PostgreSQL
python -m scripts.seed # (opcjonalnie) dane przykładowe
uvicorn app.main:app --reload # API na http://localhost:8000

# Frontend (w osobnym terminalu)
cd car-rental-frontend
npm install
npm run dev # aplikacja na http://localhost:3000
```

Uruchomienie testów:

```
cd car-rental-backend && pytest           # backend (pytest)
cd car-rental-frontend && npm run test     # frontend (Jest)
cd e2e && npx playwright test             # end-to-end (wymaga działającej
↪ aplikacji)
```

17. Spis listingów i rysunków

Listingi: 1 — model `User`/role; 2 — model pojazdu; 3 — generowanie JWT; 4 — logowanie; 5 — `require_roles`; 6 — mnożnik ryzyka; 7 — wycena; 8 — rdzeń scoringu ryzyka; 9 — finalizacja zwrotu; 10 — tworzenie recenzji; 11 — mail potwierdzenia; 12 — middleware tras; 13 — `AuthContext`; 14 — hook wyceny (SWR); 15 — mutacja recenzji + inwalidacja; 16 — kontekst motyw/język; 17 — hook tłumaczeń; 18 — filtrowanie nawigacji po roli; 19 — walidacja hasła.

Rysunki: 1 — ERD PostgreSQL; 2 — ERD MongoDB; 3 — Redis (namespace); 4 — katalog pojazdów; 5 — logowanie; 6 — mail weryfikacyjny (Mailpit); 7 — podsumowanie ceny; 8 — karta klienta (ryzyko/incydenty); 9 — zwrot pojazdu; 10 — recenzje; 11 — moderacja recenzji; 12 — mail potwierdzenia (Mailpit); 13 — ustawienia (dark + PL); 14 — pulpit pracownika; 15 — panel serwisanta; 16 — rejestracja (siła hasła); 17 — Swagger UI; 18 — kreator rezerwacji; 19 — historia wynajmów; 20 — statusy CI.